Skript zur Veranstaltung

Automaten, Sprachen und Compiler (INF2202)

Michael Elberfeld

Sommersemester 2026



Version der Unterlagen vom 2026-05-12

Inhalte auf einen Blick

I Implementierung und Entwurf von Programmiersprachen 2

1 Implementierung von Programmiersprachen 3

2 Entwurf der Tiny Educational Language (TiEL) 13

II Lexikalische Analyse von Programmiersprachen 23

3 Scannen von Programmen in TiEL 24

4 Scannergenerierung: Reguläre Ausdrücke auswerten 35

5 Scannergenerierung: Automaten simulieren 46

III Syntaktische Analyse von Programmiersprachen 58

6 Parsen von Programmen in TiEL 59

Parsergenerierung: Grammatiken transformieren 68

Parsergenerierung: Kellerautomaten simulieren 68

IV Auswerten von Programmiersprachen 69

Semantische Analyse von Programmen in TiEL 70

Auswerten von Programmen in TiEL 70

V Theorie der Implementierung von Programmiersprachen 70

Scanner- versus Parsergenerierung 71

Top-Down- versus Bottom-Up-Parsing 71

Syntaktische versus semantische Analyse 71

Zusammenfassung und Ausblick 71

Inhaltsverzeichnis

I Implementierung und Entwurf von Programmiersprachen 2

1 Implementierung von Programmiersprachen 3
1.1 Motivation . . . 4
1.1.1 Nutzung existierender Sprachen . . . 4
1.1.2 Entwicklung »eigener« Sprachen . . . 4
1.2 Sprachimplementierung mit Compilern . . . 5
1.2.1 Quellsprache Übersetzen . . . 5
1.2.2 Zielsprache Ausführen . . . 7
1.3 Sprachimplementierung mit Interpretern . . . 8
1.3.1 Quellsprache Ausführen . . . 8
1.3.2 Fragestellungen . . . 9
1.4 Der TiEL-Interpreter . . . 9
1.4.1 Verwendung . . . 9
1.4.2 Implementierung . . . 10
1.5 Zusammenfassung . . . 11
Aufgaben mit Lösungen . . . 11

2 Entwurf der Tiny Educational Language (TiEL) 13
2.1 Entwurf von Programmiersprachen . . . 13
2.2 Entwurf von TiEL . . . 15
2.2.1 Programmstruktur . . . 15
2.2.2 Datentypen und Bezeichner . . . 16
2.2.3 Namensräume und Funktionen . . . 17
2.2.4 Ausdrücke und Aufrufe . . . 18
2.2.5 Anweisungen . . . 19
2.3 Verwendung von TiEL . . . 20
2.4 Erweiterungen von TiEL . . . 21
2.5 Zusammenfassung . . . 21

II Lexikalische Analyse von Programmiersprachen 23

3 Scannen von Programmen in TiEL 24
3.1 Lexikalische Analyse . . . 24

3.2 TiELs Grundsymbole . . . 25
3.3 Lexikalische Analyse im TiEL-Scanner . . . 27
3.4 Grundsymbole mit regulären Ausdrücken beschreiben . . . 28
3.4.1 Reguläre Ausdrücke . . . 28
3.4.2 Reguläre Ausdrücke in der Praxis . . . 29
3.5 Lexikalische Analyse mit Scannergeneratoren . . . 30
3.5.1 Von Ausdrücken zum Scanner . . . 30
3.5.2 Scannergenerator JFlex . . . 30
3.6 Zusammenfassung . . . 31
Aufgaben mit Lösungen . . . 32

4 Scannergenerierung: Reguläre Ausdrücke auswerten 35
4.1 Architektur von Scannergeneratoren . . . 36
4.2 Automaten für reguläre Ausdrücke . . . 37
4.2.1 Nichtdeterministische endliche Automaten . . . 37
4.2.2 Konstruktion von Thompson . . . 38
4.3 Eliminierung von Epsilon-Übergängen . . . 39
4.4 Kontext: Reguläre Ausdrücke für Automaten . . . 40
4.4.1 Transformation am Beispiel . . . 41
4.4.2 ∗ Allgemeine Transformation . . . 42
4.5 Zusammenfassung . . . 43
Aufgaben mit Lösungen . . . 44

5 Scannergenerierung: Automaten simulieren 46
5.1 Motivation . . . 47
5.2 Von Nichtdeterminismus zu Determinismus . . . 47
5.2.1 Deterministische endliche Automaten . . . 48
5.2.2 Teilmengenkonstruktion . . . 49
5.3 Minimierung deterministischer Automaten . . . 51
5.3.1 Zustandsäquivalenz . . . 51
5.3.2 Minimierungsalgorithmus . . . 52
5.4 Zusammenfassung . . . 53
Aufgaben mit Lösungen . . . 53

III Syntaktische Analyse von Programmiersprachen 58

6 Parsen von Programmen in TiEL 59
6.1 Syntaktische Analyse . . . 59
6.2 TiELs Syntax . . . 60
6.2.1 TiELs konkrete Syntax . . . 60
6.2.2 TiELs abstrakte Syntax . . . 60
6.3 Syntaktische Analyse im TiEL-Parser . . . 61
6.4 Syntax mit Grammatiken beschreiben . . . 62
6.4.1 Kontextfreie Grammatiken . . . 62
6.4.2 Präzedenz und Assoziativität . . . 64

6.5 Syntaktische Analyse mit Parsergeneratoren 65
6.5.1 Von Grammatiken zum Parser 65
6.5.2 Scannergenerator Cup 66
6.6 Zusammenfassung 66
Aufgaben mit Lösungen 67

Parsergenerierung: Grammatiken transformieren **68**

Parsergenerierung: Kellerautomaten simulieren **68**

IV Auswerten von Programmiersprachen **69**

Semantische Analyse von Programmen in TiEL **70**

Auswerten von Programmen in TiEL **70**

V Theorie der Implementierung von Programmiersprachen **70**

Scanner- versus Parsergenerierung **71**

Top-Down- versus Bottom-Up-Parsing **71**

Syntaktische versus semantische Analyse **71**

Zusammenfassung und Ausblick **71**

Teil I

Implementierung und Entwurf von Programmiersprachen

Kapitel 1

Implementierung von Programmiersprachen

1.1 Motivation 4
1.1.1 Nutzung existierender Sprachen 4
1.1.2 Entwicklung »eigener« Sprachen 4
1.2 Sprachimplementierung mit Compilern 5
1.2.1 Quellsprache Übersetzen 5
1.2.2 Zielsprache Ausführen 7
1.3 Sprachimplementierung mit Interpretern 8
1.3.1 Quellsprache Ausführen 8
1.3.2 Fragestellungen 9
1.4 Der TiEL-Interpreter 9
1.4.1 Verwendung 9
1.4.2 Implementierung 10
1.5 Zusammenfassung 11
Aufgaben mit Lösungen 11

Höhere Programmiersprachen, wie Java oder Python, bieten Konstrukte, die nicht unmittelbar auf Maschinenbefehlen basieren. Programme in diesen Sprachen werden durch Compiler in maschinennahe Programme übersetzt oder durch Interpreter ausgeführt. Compiler und Interpreter verarbeiten gegebene Programme in der Regel in aufeinanderfolgenden Phasen; angefangen mit einer Textdatei und hin zu einer internen abstrakten Repräsentationen des Programms. Die Aufteilung in Phase, die wir in diesem Kapitel betrachten, führt zu (1) einfachen, (2) effizienten und zudem (3) portablen Implementierungen von Programmiersprachen.

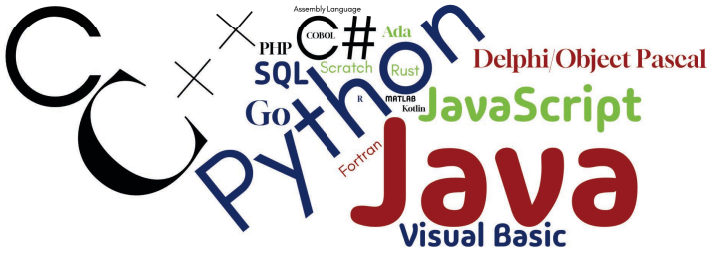
Lernziele

1. *Anwendungen* für den Entwurf und die Implementierung von Programmiersprachen kennen.
2. Funktionsweise und Architektur von *Compilern* und *Interpretern* kennen.
3. *Lernziele* und *Inhalte* dieser Veranstaltung kennen.

1.1 Motivation

1.1.1 Nutzung existierender Sprachen

Kenntnis des Entwurfs von Sprachen hilft bei deren Auswahl



Own work, Created using WordClouds.com based on popularity statistics from the TIOBE Index.

Frage

Wovon hängt die Wahl einer Programmiersprache für eine Aufgabe ab?

Antwort

Entwurfsprinzipien der Sprache; beispielsweise bezüglich Typisierung oder Verwaltung des dynamischen Speichers.

Weitere Antworten

Anwendungsgebiete der Sprache abhängig von dessen Ökosystem. Verbreitung und Community der Sprache.

Kenntnis der Sprachimplementierung hilft bei Verwendung

Fragen

Welcher Programmcode führt zu *effizienten Anwendungen*? Was bedeuten die *Fehlermeldungen* bei der Verarbeitung? Wie liest man die Ausgabe eines *Debuggers*?

```
Breakpoint 1, main () at test.c:8
8               int a = 42;
(gdb) bt
#0  main () at test.c:8
(gdb) info frame
Stack level 0, frame at 0x7fffffffe850:
 rip = 0x40060d in main (test.c:8); saved rip 0x7ffff7a77ead
 source language c.
 Arglist at 0x7fffffffe840, args:
 Locals at 0x7fffffffe840, Previous frame's sp is 0x7fffffffe850
 Saved registers:
  rbp at 0x7fffffffe840, rip at 0x7fffffffe848
(gdb) l
3       # include "../headers/liblist_single.h"
4
5       int main (void)
6       {
7               void* ce;
8               int a = 42;
9               int b = 1337;
10              int c = 314;
11
12              struct list_s* list = create_s('s'); // Our data
(gdb) n
9               int b = 1337;
(gdb) n
10              int c = 314;
(gdb) n
12              struct list_s* list = create_s('s'); // Our data
(gdb) s
create_s (location=115 's') at liblist_single.c:40
40              if ( location != 'h' && location != 's' )
(gdb) bt
```

License: CC BY-SA 3.0, Content: ebugging session with gdb 7.3, Author: Der Messer, cropped

Antwort

Es hängt von der Implementierung der Sprache ab.

1.1.2 Entwicklung »eigener« Sprachen

»Ihre Sprache« ist meist Teil einer Anwendung

Definition: Anwendungssprache

Eine *Anwendungssprache* (alt. domänenspezifische Sprache) ist für einen bestimmten abgegrenzten Zweck konzipiert.

Beispiel: Dateiformate als Anwendungssprachen
XML- und JSON-Schemata sind domänenspezifisch.

Beispiel: Schnittstellen als Anwendungssprachen
Kommandozeilenschnittstellen (kurz: CLI) sind domänenspezifisch.

Beispiel: Skriptsprachen als Anwendungssprachen
Serveranwendungen arbeiten häufig domänenspezifischen Code als Eingabe ab; beispielsweise für die Datenabfrage mit SQL.

Bemerkung
Bei der Entwicklung »Ihrer Sprache« hilft Ihnen die Kenntnis von Entwurfsprinzipien für Sprachen.

»Ihre Sprache« muss ebenfalls implementiert werden

Systematiken und Werkzeuge
Architekturprinzipien und *Algorithmen* für die Implementierung *Werkzeuge* zur Generierung wiederkehrender Programmieraufgaben. Spezifikation von Programmstruktur durch *Grammatiken* und Programmeinheiten durch *reguläre Ausdrücke*.

Bemerkung
Systematik hilft Ihnen bei der Implementierung »Ihrer Sprache«. Werkzeuge kürzen die Implementierung »Ihrer Sprache« ab.

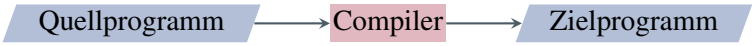
1.2 Sprachimplementierung mit Compilern

1.2.1 Quellsprache Übersetzen

Compiler übersetzen Programme

Definition
Zur *Kompilierzeit* wird der *statische* Programmtext ohne Eingaben verarbeitet.

Definition
Ein *Compiler* übersetzt ein Programm der *Quellsprache* in ein *gleichbedeutendes* Programm der *Zielsprache*.



Diskussion
Was bedeutet »gleichbedeutend«?
Um das Konzept gleichbedeutender (alt. äquivalenter) Programme zu fassen, wird in der Regel eine (abstrakte) Semantik für sowohl die Quell- als auch die Zielsprache definiert. Dies basiert häufig auf einer (abstrakten) Maschine, die angibt, wie man ein Programm durch Operationen ausführt. Erzeugen beide Programme für jede mögliche Eingabe gleiche Ausgaben, so sind die Programme *gleichbedeutend*. Eine abstrakte Maschine berücksichtigt dabei nicht praktische Beschränkungen, wie beispielsweise den zur Verfügung stehenden Speicher. Sie ist Teil der *Sprachdefinition*, auf dessen Basis sich verschiedene Compiler (wie beispielsweise `gcc` und `clang` für C) implementieren

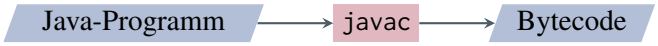
lassen, die das gleiche Ein-Ausgabeverhalten haben. Eine knappe visuelle Darstellung dieser Konzepte findet sich in Abbildung 1.1-1 Seite 14 im referenzieren Buch von Kastens.

Beispiel: Übersetzung von hoher zu niedriger Sprachebene

- Übersetzung in Instruktionen einer CPU:



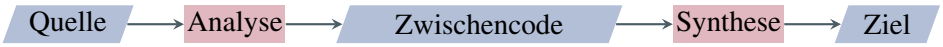
- Übersetzung zur Sprache einer virtuellen Maschine:



Compiler »verstehen« Quellcode, um Zielcode zu erzeugen

Definition

Die *Analyse* liest das Quellprogramm und erkennt dessen Struktur. Die *Synthese* schreibt das Zielprogramm auf Basis der erkannten Struktur. Programmstruktur wird durch *Zwischencode* repräsentiert.

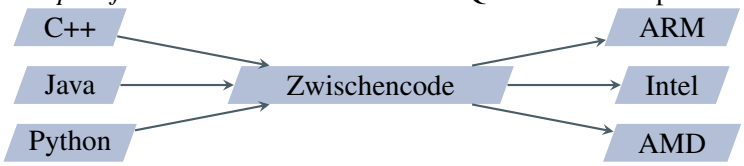


Bemerkung

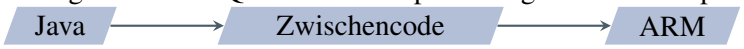
Analyse hängt von Quellsprache ab, aber nicht von Zielsprache. Synthese hängt von Zielsprache ab, aber nicht von Quellsprache. Gemeinsame Zwischensprache ermöglicht Kombination.

Beispiel: Portierung von Compilern mit Zwischencode

- *Compilerframework* mit verschiedenen Quell- und Zielsprachen:



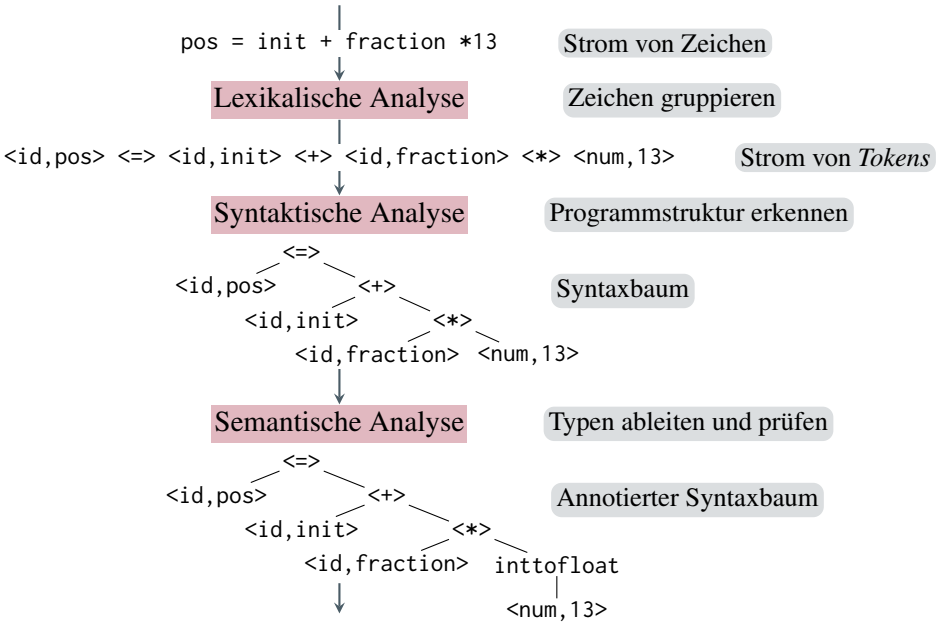
- Konfiguration von Quell -und Zielsprache ergibt einen Compiler:



Die LLVM Compiler Infrastructure verwendet beispielsweise die *LLVM intermediate representation* und bietet eine Synthese für viele Prozessoren.

Phasen der Analyse »verstehen« das Programm schrittweise

Pipeline der Analyse mit Zwischenergebnissen



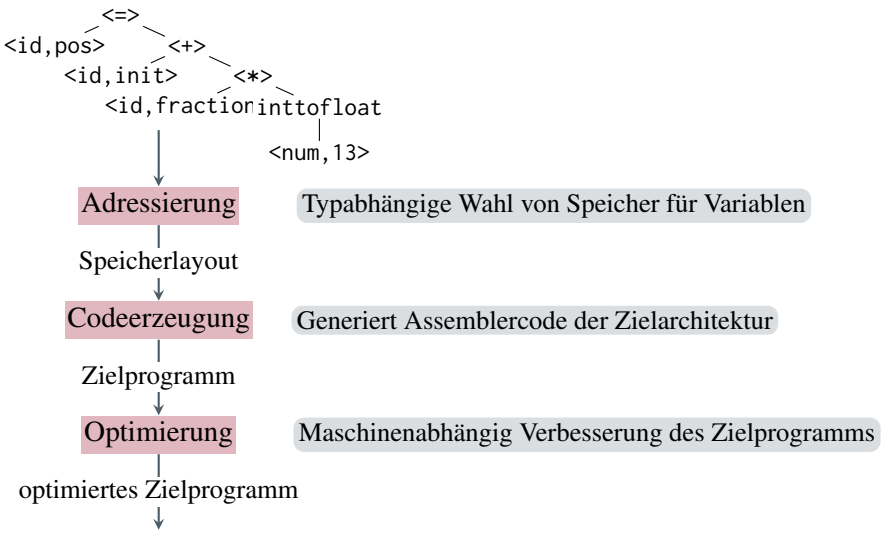
Diskussion

Weitere Phase optimiert Programm *maschinenunabhängig*. Was könnte man verbessern? Ergibt dies ein optimales Programm?

Wir möchten in der Regel eine Verbesserung der Laufzeit und des Speicherbedarfs des übersetzten Programms erreichen. Ideen hierzu sind vielfältig. Klassisch wird nicht erreichbarer Code aus dem Programm gelöscht, invariante Ausdrücke aus einer Schleife vor diese verschoben und konstante Variablen bei deren Anwendung durch ihren Werte ersetzt. Dies sind Verbesserung, die in der Regel nicht zu einem global optimalen Programm führen, sondern lokale Verbesserungen darstellen.

Synthese generiert das Zielprogramm ebenfalls in Phasen

Pipeline der Synthese mit Zwischenergebnissen



1.2.2 Zielsprache Ausführen

Programm in Zielsprache wird geladen und ausgeführt

Definition

Zur *Laufzeit* wird das Zielprogramm für *dynamische* Eingaben ausgewertet.

Beispiel: Ausführung auf einem Prozessor

(1) Maschinencode *laden* und an Abhängigkeiten *binden*. (2) Ausführung für Eingabe auf einem Prozessor starten.

Beispiel: Ausführung durch eine virtuelle Maschine

(1) Bytecode an virtuelle Maschine übergeben. (2) Virtuelle Maschine führt aus und compiliert gegebenenfalls nach.

Die Java HotSpot Performance Engine ist beispielsweise eine virtuelle Maschine, die Java Bytecode ausführt und Programmteile, die häufig verwendet werden, nachträglich in Maschinencode für die jeweils verwendete Hardware übersetzt.

Bemerkung

Virtuelle Maschinen sind Inhalt der Veranstaltung Anwendung systemnaher Konzepte in der Programmierung (INF2201). Wir konzentrieren uns auf Quellsprachen auf höherer Abstraktionsebene.

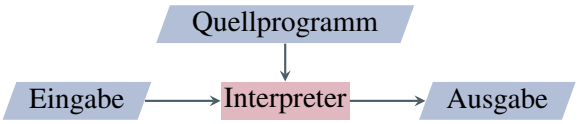
1.3 Sprachimplementierung mit Interpretern

1.3.1 Quellsprache Ausführen

Interpreter sind virtuelle Maschinen für die Quellsprache

Definition

Ein *Interpreter* führt ein gegebenes Programm der *Quellsprache* für gegebene *Eingaben* korrekt aus.



Bemerkung

Korrekte Ausführung bedeutet, dass das Ein-Ausgabeverhalten für alle möglichen Eingeben der Spezifikation der Sprache in der Sprachdefinition entsprechen muss. Interpreter sind häufig eine Implementierung der in der Sprachdefinition verwendeten abstrakten Maschine und ähneln von ihren Komponenten während der Auswertung eine Prozessor, der neben einem Programm auch die Daten hält. Insbesondere sollte kein gültiges Programm der Quellsprache einen Fehler verursachen. Umgekehrt sollte jedes Programm, welches fehlerhaft ist, zu einer Fehlermeldung im Interpreter führen.

Beispiel: Interpretation von Sprachen auf hoher Abstraktionsebene

JavaScript wird im Browser interpretiert. Python wird interpretiert mittels einer Transformation in Bytecode.

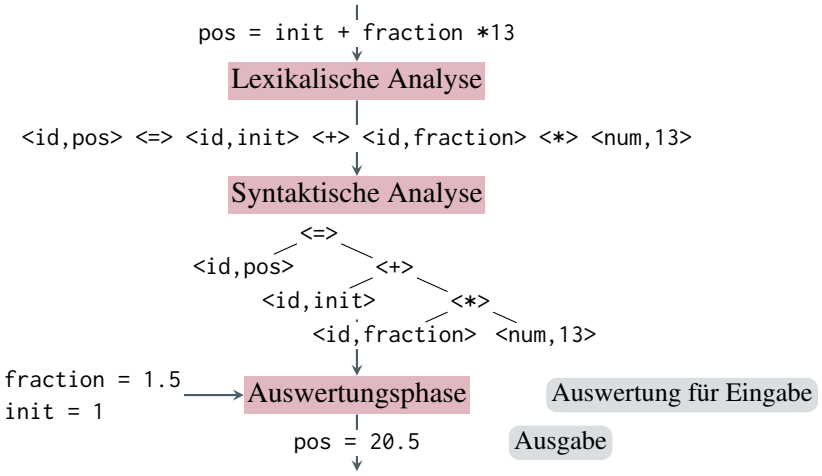
Bemerkung

Pro *Interpreter erlauben höhere Portierbarkeit, genauere Fehlerdiagnose und kürzere Entwicklungszeiten, aber*

Contra *sind in der Regel deutlich langsamer.*

Interpreter verwenden weniger und teils andere Phasen

Pipeline eines Interpreters ohne semantische Analyse



1.3.2 Fragestellungen

Dieses Veranstaltung beschäftigt sich mit Interpretern

Bemerkung

Interpreter werden häufig für Anwendungssprachen verwendet und sind konzeptuell einfacher als Compiler.

Inhalt I: Interpreter im Allgemeinen

Konzepte der lexikalischen und syntaktischen Analyse. Dynamische Auswertung auf Basis eines Zwischencodes.

Inhalt II: Interpreter im Konkreten

Architektur und Funktionsweise eines konkreten Interpreters. Selbstständige Erweiterung des Interpreters für Spracherweiterungen.

1.4 Der TiEL-Interpreter

1.4.1 Verwendung

TiEL ist eine für Erweiterungen offene Kernsprache

Bemerkung

Die Quellsprache dieser Veranstaltung ist eine »winzige« dynamisch getypte Sprache, die Tiny Educational Language (TiEL).

Beispiel: Fibonacci-Zahlen in TiEL

```
1 fun fib(n) {
2   if n == 0 {
3     return 0;
4   } elif n == 1 { // elif means else if
5     return 1;
6   } else {
```

```
7     return fib(n - 1) + fib(n - 2);
8   }
9 }
10
11 print(fib(12));
```

Programm `fibonacci.tiel` im Verzeichnis `examples`.

1.4.2 Implementierung

Der TiEL-Interpreter ist ein Java-Programm

Definition

Die *Implementierungssprache* ist die Sprache, in der ein Compiler oder Interpreter implementiert ist.

Bemerkung

Der Interpreter dieser Veranstaltung ist in Java implementiert. Entwickelt wird er in einem IntelliJ IDEA Projekt mit Gradle.

Warum Java?

Ist (gegenüber C) auf einer so hohen Abstraktionsebene, dass wir uns nicht mit Details der Hardware beschäftigen müssen. Ist (gegenüber Python) auf einer so niedrigen Abstraktionsebene, dass wir Algorithmen und Datenstrukturen im Detail kontrollieren können.

Der TiEL-Interpreter kommt ohne semantische Analyse aus

Definition

Der *Scanner* (alt. Lexer) ist die Komponente, die lexikalische Analyse durchführt.

Definition

Der *Parser* ist die Komponente, die die syntaktische Analyse durchführt.

Definition

Der *Auswerter* (alt. Interpreter) ist die Komponente, die die auswertet beziehungsweise ausführt.

Interpreter-Phasen in TiEL

```
    var tokens = new Scanner(src).scan();
printTokens(tokens);
    var ast = new Parser(tokens).parse(); printAST(ast);
    var evaluator = new
Evaluator(Globals.initEnv(System.out));
    new Binder(evaluator).resolve(ast);
    evaluator.interpret(ast);
```

Klasse `Main`, Methode call()

1.5 Zusammenfassung

- Programmiersprachen werden *entworfen* und *implementiert*. Der *Entwurf* legt die *syntaktischen* und *semantischen* Eigenschaften der Sprache fest.
- Die *Sprachimplementierung* legt fest, wie Programme der Sprache verarbeitet werden.
- *Compiler* sind Sprachimplementierungen, die ein gegebenes *Quellprogramm* in der *Quellsprache* in ein *Zielprogramm* in der *Zielsprache* übersetzen. Das Zielprogramm wird danach auf der Hardware oder virtuell ausgeführt.
- *Interpreter* sind Sprachimplementierungen, die ein gegebenes *Quellprogramm* für gegebene *Eingaben* »direkt« ausführen, wobei häufig intern übersetzt wird.
- Ein Compiler ist konzeptionell in *Analyse* und *Synthese* aufgeteilt, wobei die Analyse die *lexikalische*, *syntaktische* und *semantische* Analyse als Phasen umfasst und die Synthese die Phasen *Adressierung*, *Codegenerierung* und *maschinenabhängige Optimierung* durchführt.
- Interpreter verwenden eine *Auswertungs- beziehungsweise Ausführungsphase*, die im Fall des TiEL-Interpreters die Synthese und die semantische Analyse ersetzt.

Literatur

[1] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021. (Kapitel 1 und 2)

[2] A. Aho, M. Lam, R. Sethi und J. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2006. (Abschnitte 1.1, 1.2 und 1.4)

[3] Hanspeter Mössenböck. *Compilerbau: Grundlagen und Anwendungen*. dpunkt.verlag, 2024. (Abschnitte 1.1 bis 1.3 sowie Anhang B)

[4] Uwe Meyer. *Grundkurs Compilerbau*. Rheinwerk Computing, 2021. (Kapitel 1)

Aufgaben

A 1 (Konzepte der Implementierung von Programmiersprachen wiedergeben, alte Klausuraufgabe)

Beantworten Sie die folgenden Fragen jeweils kurz und eindeutig.

1. In welche 3 Phasen teilt sich die Verarbeitung von Programmen durch Interpreter auf.
2. Welche Phase des Interpreters generiert Token.
3. Aus welchen 3 Teilen besteht das Token für ein Literal.

Lösungen

A 1

1. 1. Lexikalische Analyse (alt. Scannen)
   2. syntaktische Analyse (alt. Parsen)
   3. Auswertungsphase
2. Lexikalische Analyse
3. - Grundsymbol
   - Lexem
   - Tokenwert

Kapitel 2

Entwurf der Tiny Educational Language (TiEL)

2.1 Entwurf von Programmiersprachen 13
2.2 Entwurf von TiEL 15
2.2.1 Programmstruktur 15
2.2.2 Datentypen und Bezeichner 16
2.2.3 Namensräume und Funktionen 17
2.2.4 Ausdrücke und Aufrufe 18
2.2.5 Anweisungen 19
2.3 Verwendung von TiEL 20
2.4 Erweiterungen von TiEL 21
2.5 Zusammenfassung 21

Wir beschäftigen uns in den kommenden Kapiteln sowohl mit allgemeinen Konzepten des Interpreter- und Compilerbaus als auch mit der konkreten Implementierung eines Interpreters. Dieser Interpreter implementiert die Tiny Educational Language (TiEL), welche für die Veranstaltung entworfen wurde. Diese Sprache ist in Ihrer Grundausstattung, wie der Name sagt, winzig. Allerdings wird Sie das nicht lange bleiben. In den Hausaufgaben erweitern Sie diese Sprache durch neue Elemente und verbessern Ihre Anwendbarkeit. Damit Sie darauf gut vorbereitet sind, finden Sie in diesem Kapitel die grundlegenden Designprinzipien von TiEL beschrieben sowie deren Vergleich mit alternativen Ansätzen.

Lernziele

1 Generationen und Paradigmen von Programmiersprachen kennen.
2 Die *Tiny Educational Language (TiEL)* kennen und Programme in TiEL *entwickeln* und *testen* können.
3 *Entwurfsprinzipien* von Programmiersprachen anhand von TiEL kennen und beschreiben können.

2.1 Entwurf von Programmiersprachen

Sprachen werden nach Generationen unterschieden



License: CC BY 3.0, Content: Entwicklung des Fahrrades mit Jahresangaben, Author: AI2, cropped, years omitted and rearranged.

Generationen von Sprachen

1st Maschinensprache wie TOY in »Theoretische Informatik 1«.
2nd Assemblersprachen wie ATmega oder 80x86.
3rd Höhere Programmiersprachen wie C, C++, Java oder Scala.
4th Anwendungssprachen wie SQL oder SVG.
5th Logikbasierte Sprachen wie Prolog oder Datalog.

Bemerkung

Entwicklung von

- *. . . maschinenlesbar zu menschenlesbar.*
- *. . . imperativ (»was zu tun«) zu deklarativ («was man erreichen möchte«).*

Sprachen kann man nach Paradigmen unterscheiden

Definition

Paradigma beschreibt Darstellung der *Daten* und Art des *Kontrollflusses*.

Paradigmen von Sprachen

imperativ Programm ist eine Folge von Zustandsänderungen: Fortran, Algol, C
funktional Ergebnis der Berechnung ist Rückgabe einer Funktion: Haskell, Clojure, LISP, Scala
objektorientiert Objekte mit Eigenschaften und Fähigkeiten: C++, Java, C#
relational Daten sind Zeilen einer Tabelle und werden verbunden: SQL, Prolog- und Datalog-Dialekte

Bemerkung

Moderne Sprachen implementieren mehrere oder abgewandelte Paradigmen. Typisch ist die Kombination objektorientierter und funktionaler Aspekte.

Sprachen kann man nach Entwurfsprinzipien unterscheiden

Definition: Syntaktische und Semantische Eigenschaften

Syntax Betrifft den *Aufbau* eines Programms aus Grundsymbolen.
Semantik Betrifft die *Bedeutung* des Programms.

Definition: Statische und dynamische Vorgaben

Statisch *Ohne Ausführung* prüfbare Vorgaben.
Dynamisch Erst *bei Ausführung* prüfbare Vorgaben.

Was wann geprüft wird hängt von der Sprachimplementierung ab.

Beispiel: Prüfung statischer Typisierung in Java zur Kompilierzeit

```
int a = 5;
var b = a; // compiler resolves type int for b
b == false; // Error: incomparable types: int and boolean
```

Beispiel: Statischer und dynamischer Speicher in Java

```
class Test {
  static int a = 5; // once allocated for whole program
  int b = 4; // allocated for each object
}
```

2.2 Entwurf von TiEL

2.2.1 Programmstruktur

TiEL ist eine »winzige« imperative Skriptsprache

Diskussion

Was ist die Ausgabe des folgenden Programms?

```
2  fun foo(flag) {
3    bar(1);
4    if flag {
5      return "World!";
6    }
7  }
8  fun bar(i) {
9    while i < 2 {
10     print("Hello"); // built-in function print
11     i = i + 1;
12   }
13 }
14 print(foo(true));
```

Programm `hello_world_extended.tiel` im Verzeichnis `examples`.

Struktur von TiEL-Programmen

Steht in einer Textdatei mit Endung `.tiel`. Das Programm ist eine *Folge von Deklarationen und Anweisungen*. Deklarationen und Anweisungen werden *schrittweise* interpretiert. *Kapselung* von Code durch Funktionen und deren *Aufrufe*. Wenige Operatoren, Konstrollstrukturen und Datentypen.

Bemerkung

In den Aufgaben entwickeln Sie Erweiterungen von TiEL. Falls Ihnen Weiteres fehlt, können Sie dies selbst implementieren.

2.2.2 Datentypen und Bezeichner

TiEL unterstützt nur eingebaute Datentypen

Definition: Datentyp
Ein *Datentyp* (kurz: *Typ*) ist ein *Datensatz* mit zugehörigen *Operationen*.

Datentypen in TiEL
Tiel unterstützt *Gleitkommazahlen*, *Wahrheitswerte* und *Strings*. Zusätzlicher Wert `nil` steht für »kein Wert vorhanden«.

Definition: Literal
Literale sind in einem festgelegten Format geschriebene Werte.

Literale in TiEL
Gleitkommazahlen wie `56.34`, Wahrheitswerte `true` und `false`, Strings wie `"T␣H␣M"` sowie `nil`.

Beispiel: Syntaktisch unterstützt TiEL ganze Zahlen

```
var a = 2; // represented as 2.0 internally
var b = 2.0;
print(a == b); // outputs true
print(b); // outputs just 2
```

Der »Punkt« muss bei Gleitkommazahlen nicht zwingend mit angegeben werden. Beispielsweise wird `4` intern so verarbeitet, als ob man es im Programmcode durch `4.0` ersetzen würde. Funktion `print` entfernt aber den Punkt, falls die Zahl keinen Nachkommateil enthält.

Deklarationen führen Namen ein

Definition: Bezeichner
Bezeichner (engl. identifier) sind im Programm verwendete Namen für beispielsweise Variablen und Funktionen.

Definition: Deklarationen und Definitionen
Deklaration führt einen Bezeichner für eine Variable oder Funktion ein. *Definition* weist einen *initialen* Wert zu.

Definition: Zuweisung
Zuweisung (engl. assignment) weist einen Wert zu.

Deklarationen sind immer auch Definitionen in TiEL

```
var a = nil; // defines variable
a = 6; // assigns value
var b; // Expected '=' after variable name, but got SEMICOLON.
```

```
fun inc(i) { return (i + 1); } // defines function
fun dec(i); // Expected '{' before function body, but got SEMICOLON.
```

Typen sind erst zur Laufzeit bekannt und veränderlich

Definition: Statische Typisierung
Variablen haben einen zur Kompilierzeit festlegbaren Typ. Typ der Variable kann sich zur Laufzeit nicht ändern.

Definition: Dynamische Typisierung
Typ von Variablen ergibt sich zur Laufzeit. Neben dem Wert kann sich auch der Typ einer Variable ändern.

TiEL als Skriptsprache ist dynamisch getypt

```
var a = 5;
if a < 4 { a = "string"; } else { a = false; }
print(a); // prints out false
```

Bemerkung

Typen dynamisch *Weniger Code, schnellere Umsetzung und Anpassung.*
Typen statisch *Weniger Fehler und effizientere Programme.*
Typisierung ist ein Teil der Semantik des Programms, welches schon in der Kompilierzeit analysiert wird; dies fängt Fehler ab. Durch statische Typisierung lassen sich die Operatoren, die auf die Werte angewendet werden schon zur Kompilierzeit auswählen. Dies beschleunigt insbesondere das von Compilern generierte Zielprogramm.

2.2.3 Namensräume und Funktionen

Gültigkeit basiert auf statischer Schachtelung

Definition
Gültigkeitsbereich einer Deklaration von x ist der Bereich im Code, in dem ein Auftreten von x auf diese Deklaration verweist.

Definition: Statischer Gültigkeitsbereich (engl. static scope)
Deklaration von x gilt in dem kleinsten umfassenden Block und textuell danach. Ausgenommen sind *verdeckende* Gültigkeitsbereiche in darin enthaltenen Blöcken für Deklarationen mit x.

Definition: Dynamischer Gültigkeitsbereich (engl. dynamic scope)
Gültigkeit von x ist analog zum statischen Geltungsbereich, bezieht sich aber auf die dynamische Schachtelung durch Funktionsaufrufe zur Laufzeit.
Es gibt alternative Definitionen für dynamische Gültigkeitsbereiche. Beispielsweise kann man in der Programmiersprache Groovy, die wie Java und Scala auf der JVM ausgeführt wird, anonymen Funktionen, sogenannten Closures, programmatisch Kontexte zuweisen, die den dynamischen Gültigkeitsbereich definieren.

Gültigkeitsbereiche in TiEL sind statisch

```
var a = 1; // declaration
fun foo(d) {
  a = 2; // refers to a in line 1
  var a = 3; // shadows a in line 1
  print(a); // refers to a in line 4; prints 3
}
foo(0);
print(a); // refers to a in line 1; prints 2
```

Namensräume beinhalten deklarierte Bezeichner zur Laufzeit

Gemeinsame Namensräume in TiEL

Variablen und Funktionen teilen sich die *Namensräume* in TiEL.

```
fun foo(x) {
    var foo = 5;
    print(foo);
}
var a = foo; // a is now of type function
var foo = 4; // Error: Identifier already declared 'foo'.
var foo = 5;
```

Lokale Deklaration von Funktionen in TiEL

- Der *globale* Namensraum umfasst das gesamte Programm.
- *Lokale* Namensräume entstehen durch Funktionsdeklarationen.
- Ergibt *Hierarchie* von Namensräumen.
- Insbesondere können Funktionen in Funktionen deklariert werden.

```
5  fun add(a) {
6    fun add_curried(b) {
7      return a + b;
8    }
9
10   return add_curried;
11 }
12
13 var x = add(1)(2);
14 print(x);
```

Programm `local_function.tiel` im Verzeichnis `examples`.

2.2.4 Ausdrücke und Aufrufe

Operatoren am besten einmal selbst ausprobieren

Definition: Operatoren

Operatoren sind Abkürzungen für eingebaute Funktionen.

Operanden für Zahlen

`-`, `+`, `*`, `/` und `<` erwarten Zahlen als Operanden.

Boolesche Operanden

`and`, `or`, `not` verarbeiten beliebige Operanden. Haben *Kurzschluss-Semantik* (engl. short-circuit) wie in Java. Operanden werden als Wahrheitswerte interpretiert, aber nur umgewandelt, falls notwendig.

```
print(nil and true); // prints nil
print(false or 2); // prints 2
print(not nil); // prints true
print(not 2); // prints false
```

Vergleichsoperator

`==` für beliebige Operanden basiert auf Javas `equals`-Methoden.

Definition: Ausdrücke und Präzedenzen

Ausdruck (engl. expression) erzeugt einen Wert durch geschachtelte Auswertung von Operatoren und Funktionen auf Argumenten. *Präzedenz* beschreibt die Bindungsstärke von Operatoren wie »Punkt- vor Strichrechnung«.

Ausdrücke und Präzedenzen in TiEL

Regeln sind analog zu denen in Java (siehe Java Dokumentation)

Funktionsaufrufen produzieren immer Rückgabewerte

Definition: Parameter

Formaler Parameter ist die in der Methodenkopf deklarierte Variable. *Aktueller Parameter* (alt. Argument) ist der beim Aufruf übergebene Wert.

Definition: Übergabetyp

Per Wert Wert wird von aktuellen in formalen Parameter kopiert.
Per Referenz Referenz auf Speicherstelle wird übergeben.

Funktionsaufrufe in TiEL

Anzahl formaler und aktueller Parameter muss gleich sein. Übergabe per Wert. Es gibt immer einen Rückgabewert. Explizit oder `nil` implizit.

```
fun foo(a) { a = a + a; }
print(foo(2)); // prints nil
print(1,2); // Error: Expected 1 arguments but got 2.
```

2.2.5 Anweisungen

Ablauf des Programme wird durch if und while strukturiert

Definition: Anweisungen

Anweisungen produzieren potentiell einen Effekt wie Änderung von Daten oder Anpassung des Kontrollflusses.

Zuweisungen und Ausdrücke sind Anweisungen in TiEL

```
var a = 0; // variable declaration statement
a = 1; // statement with assignment expression
a + a; // statement with binary expression
print(a); // statement with call expression producing "side effect"
```

Definition: Kontrollfluss

Der *Kontrollfluss* ist die vom Kontext abhängige Abfolge von Operationen; typischerweise in einem imperativen Programm.

Beispiel: Bedingte Anweisung mit `if-then-else` *und* `elif` *in TiEL*

```
if not false {
  1;
} elif true {
  5;
}
```

Beispiel: Schleifen mit `while-do` *in TiEL*

```
var i = 0;
while -4 < i {
  print("tick");
  i = i - 2;
}
```

2.3 Verwendung von TiEL

TiEL-Programme kann man in der Kommandozeile ausführen

Ausführung ohne Eingabedatei erzeugt Hilfe für den TiEL-Interpreter

```
java -jar TiEL.jar // calling TiEL interpreter without input

missing required parameter: '<input>'
Usage: tiel [-hV] [--ast] [--tokens] <input>
      <input>     Source code file to execute.
      --ast       Print generated AST.
  -h, --help      Show this help message and exit.
      --tokens    Print scanned tokens.
  -V, --version   Print version information and exit.
```

Ausführung mit Eingabedatei erzeugt Ausgabe

`java -jarTiEL.jar fibonacci.tiel` führt das Programm aus Kapitel 1 aus und schreibt Fibonaccizahlen auf die Standardausgabe.

Unit-Tests ermöglichen schnelle und korrekte Entwicklung

Definition
Bei der *testgetriebene Entwicklung* (engl. test-driven development, TDD) erstellt man vorab Tests für jeden zu implementierenden Programmteil.

Definition
Regressionstest prüft, ob Änderungen Funktionalität beibehält.

JUnit-Test im TiEL-Interpreter

```
@Test
void evaluatesArithmeticAssignmentAndWhileLoop() {
  var output = evaluate("""
      var␣x␣=␣1;
      while␣x␣<␣5␣{␣x␣=␣x␣+␣1;␣}
      print(x);␣print(3␣+␣4);""");
  assertEquals("""
      5
      7
      """, output);
}
```

Klasse `EvaluatorTest`, Klasse EvaluatorTest

2.4 Erweiterungen von TiEL

»TiEL kann viel«, aber auch »TiEL fehlt viel«

Weitere Operatoren und eingebaute Funktionen

- Operatoren wie <= für Zahlen oder + für Strings.
- Funktionen für Teilstrings und lesen von der Standardeingabe.

Definierte Datentypen und Objekte

- Array und definierbare Datentypen.
- Objekte mit Methoden und Instantiierung.
- Benötigt Übergabe per Referenz (explizit oder implizit).

Funktionale Semantik und anonyme Funktionen

- Anweisung mit Rückgabewerten; `if-then-else` als Operator wie `?:` in Java.
- Anonyme Funktionen (»Lambdas«) und Kontextwechsel (»Closures«).

2.5 Zusammenfassung

- Programmiersprachen unterscheidet man nach *Generationen* und *Paradigmen*.
- Eigenschaften von Programmtexten sind *syntaktisch* oder *semantisch* beziehungsweise *statisch* oder *dynamisch*.
- *TiEL* ist eine dynamisch getypte imperative Skriptsprache mit einer »winzigen«, dafür (in Aufgaben) stark erweiterbaren, Funktionalität bezüglich Umfang an Datentypen, Operatoren und Kontrollanweisungen.
- *Entwurf* einer Sprache uns dessen *Implementierungen* sind miteinander verschränkte Aufgaben, sollten aber aus Sicht der Benutzung möglichst getrennt sein.

Literatur

[1] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021. (Kapitel 2)

[2] A. Aho, M. Lam, R. Sethi und J. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2006. (Abschnitte 1.3 und 1.6)

[3] Uwe Meyer. *Grundkurs Compilerbau*. Rheinwerk Computing, 2021. (Kapitel 2)

Teil II

Lexikalische Analyse von Programmiersprachen

Kapitel 3

Scannen von Programmen in TiEL

3.1 Lexikalische Analyse 24
3.2 TiELs Grundsymbole 25
3.3 Lexikalische Analyse im TiEL-Scanner 27
3.4 Grundsymbole mit regulären Ausdrücken beschreiben 28
3.4.1 Reguläre Ausdrücke 28
3.4.2 Reguläre Ausdrücke in der Praxis 29
3.5 Lexikalische Analyse mit Scannergeneratoren 30
3.5.1 Von Ausdrücken zum Scanner 30
3.5.2 Scannergenerator JFlex 30
3.6 Zusammenfassung 31
Aufgaben mit Lösungen 32

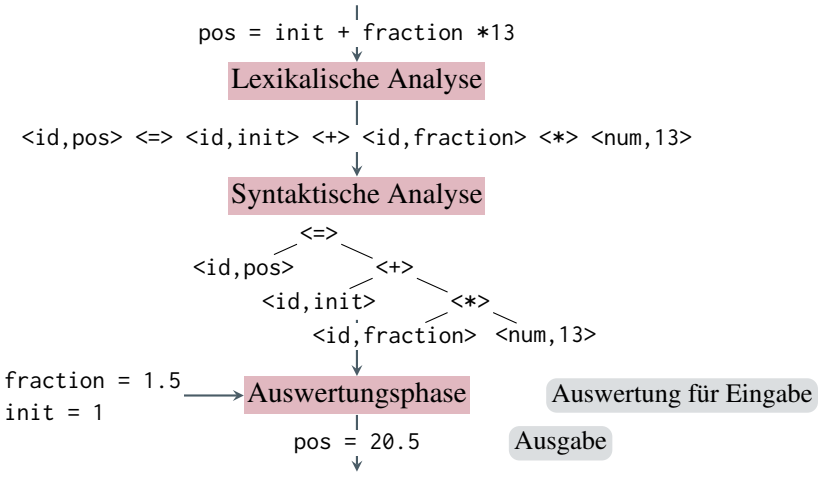
Lernziele

1 Den *TiEL-Scanner* als Beispiel eines »handgeschriebenen« Scanners kennen und anpassen können.
2 *Lexikalische Grundsymbole* von Sprachen durch *reguläre Ausdrücke* beschreiben können.
3 *Scannergeneratoren* als Alternative zu »handgeschriebenen« Scannern kennen.

3.1 Lexikalische Analyse

Lexikalische Analyse ist die erste Phase des Interpreters

Pipeline eines Interpreters



Der Scanner erkennt die Grundsymbole eines Programms

Aufgabe der lexikalischen Analyse beziehungsweise des Scanners
Grundsymbole *erkennen* und *kodieren* sowie Füllzeichen *ausblenden*.

Beispiel: TiEL-Programm als Eingabe aus Zeichen

```
if 1 or false { print("true"); } // ein Kommentar
```

Beispiel: Ausgabe des TiEL-Scanners mit Grundsymbolen

```
TOKEN(IF, if, null) @ line 1, column 1
TOKEN(NUMBER, 1, 1.0) @ line 1, column 4
TOKEN(OR, or, null) @ line 1, column 6
TOKEN(FALSE, false, null) @ line 1, column 9
TOKEN(LEFT_BRACE, {, null) @ line 1, column 15
TOKEN(IDENTIFIER, print, print) @ line 1, column 17
TOKEN(LEFT_PAREN, (, null) @ line 1, column 22
TOKEN(STRING, "true", true) @ line 1, column 23
TOKEN(RIGHT_PAREN, ), null) @ line 1, column 29
TOKEN(SEMICOLON, ;, null) @ line 1, column 30
TOKEN(RIGHT_BRACE, }, null) @ line 1, column 32
TOKEN(EOF, , null) @ line 1, column 50
```

3.2 TiELs Grundsymbole

Grundsymbole werden vom Scanner durch Token kodiert

Definition
Ein *Token* besteht aus:

- *Grundsymbol* (engl. token type) ist ein Symbol der abstrakten Syntax.
- *Lexem* ist die Folge von Zeichen eines Grundsymbols im Programm.
- *Tokenwert* (engl. token value) ist der optionale getypte Wert des Lexems.

Token im TiEL-Interpreter

```
public record Token(
    TokenType type,
    String lexeme,
    Object value,
    Position position) { // line number used for logging
```

Klasse Token, Signatur des Records

Zeichen mit Bedeutung werden zu Grundsymbolen

Definition

- *Bezeichner* (engl. identifier) sind im Programm deklarierte Namen.
- *Spezialsymbole* sind Symbole wie +, ==, ; und =.
- *Wortsymbole* (engl. keywords) sind Namen wie **while** oder **if**.
- *Literale* beschreiben einen Wert eines Datentyps der Sprache.

Tokentypen im TiEL-Interpreter

```
public enum TokenType {
  IDENTIFIER,   // Variable or function name

  // Special symbols
  LEFT_PAREN,   // (
  RIGHT_PAREN,  // )
  LEFT_BRACE,   // {
  RIGHT_BRACE,  // }
  COMMA,     // ,
```

Klasse TokenType, Aufzählungswerte Grundsymbole

Zeichen ohne Bedeutung werden ausgeblendet



License: CC BY-SA 3.0, Content: Sprengung eines Schornsteins der ehemaligen Brauerei Henninger in Frankfurt/Main, Author: Heptagon.

Definition

Füllzeichen (engl. whitespace) wie Leerzeichen, Tabulatoren und Zeilenumbrüche trennen Grundsymbole.

Definition

Kommentare wie der Folgende tragen keine Bedeutung:

```
var init = 1; // setze Startwert
```

Diskussion

Welche Bedeutung tragen Leerzeichen und Zeilenumbrüche in Python?

3.3 Lexikalische Analyse im TiEL-Scanner

Regeln bestimmen Zuordnung von Lexemen zu Symbolen

Regel für Bezeichner

Lexeme vom Typ `TokenType#IDENTIFIER` enthalten englische Buchstaben, Dezimalziffern und `'_'`; sie starten mit einem Buchstaben oder `'_'`. Ausgeschlossen sind Lexeme der *reservierten* Wortsymbole.

Regel für Spezial- und Wortsymbole

Lexeme von Spezial- und Wortsymbolen werden zum jeweiligen Grundsymbol. Beispielsweise wird == zu `TokenType#EQUAL_EQUAL` und **fun** zu `TokenType#FUN`.

Regel für Literale

`TokenType#NUMBER`-Lexeme starten mit einer Dezimalziffer. Danach folgt eine beliebige Anzahl von Dezimalziffern; optional mit Dezimalpunkt. `TokenType#String`-Lexeme starten mit `"` und enden beim nächsten `"`; dazwischen stehen beliebige Zeichen, auch Zeilenumbrüche.

Regeln werden iterativ durch Fallunterscheidung angewendet

Scanner-Algorithmus: Idee



License: CC BY-SA 3.0 DE, Content: Bahlsen Werkstor, Graffiti mit Krümelmonster und Leibnizkeks, Author: Axel Hindemith, cropped

Advance Eingabe wird *zeichenweise verarbeitet*, um Lexeme zu finden.
Add Token Wird ein Lexem gefunden, wird ein zugehöriges *Token erstellt*.
Peek Gegebenenfalls wird auf Zeichen ohne Verarbeitung *vorgegriffen* (engl. look ahead).

Scanner-Algorithmus: TiEL-Scanner

```
public List<Token> scan() {
  while (!isAtEnd()) {
    start = current;
    nextToken();
  }
  tokens.add(new Token(EOF, "", null, new
 Token.Position(line, column + 1)));
  return tokens;
}
```

Klasse Scanner, Methode scan()

```
private void nextToken() {
  var c = advance();
  switch (c) {
```

```
    case '(' -> addToken(LEFT_PAREN);
    // 7 lines omitted
    case '+' -> addToken(PLUS);
    case '=' -> addToken(match('=') ? EQUAL_EQUAL : EQUAL);
    case '<' -> addToken(LESS);
    case '/' -> {
      if (match('/')) {
        while (peek() != '\n' && !isAtEnd()) advance();
      } else {
        addToken(SLASH);
      }
    }
    case '␣', '\r', '\t' -> {} // Ignore whitespace
    // 14 lines ommitted
```

Klasse Scanner, Methode nextToken()

Der Scanner erkennt und berichtet lexikalische Fehler

Bemerkung
Programmcode in Entwicklung ist meistens »kaputt«. Fehlermeldungen und Fehlerbehandlung sind wichtige Schnittstellen zwischen Sprachimplementierung und Entwickelnden.

Grundlegende Fehlerbehandlung im TiEL-Scanner

```
      if (isDigit(c)) {
        number();
      } else if (isAlpha(c)) {
        identifier();
      } else {
        throw new ScanningError("Unexpected␣char.", new
Token.Position(line, column));
      }
```

Klasse Scanner, Method nextToken()

3.4 Grundsymbole mit regulären Ausdrücken beschreiben

3.4.1 Reguläre Ausdrücke

Lexeme eines Grundsymbols bilden eine formale Sprache

Definition: Formale Sprachen
Ein *Alphabet* Σ (»Sigma«) ist eine endliche nicht-leere Menge. Ein *Wort* w über Σ ist eine Folge von Zeichen aus Σ. Die *Länge* von w wird mit $|w|$ bezeichnet. Das *leere Wort* mit Länge 0 wird mit λ (»lambda«) bezeichnet. Eine *Sprache* L ist eine Menge von Wörtern.

Beispiel: Sprache von TiEL-Bezeichnern
Alphabet $\Sigma = \{\texttt{a}, \ldots, \texttt{z}, \texttt{A} \ldots \texttt{Z}, \texttt{0}, \ldots, \texttt{9}, _\}$. $L_{\texttt{IDENTIFIER}} = \{_, \texttt{temp3}, \ldots\}$ ist die Sprache der TiEL-Bezeichner. λ ist nicht in $L_{\texttt{IDENTIFIER}}$.

Reguläre Ausdrücke beschreiben Sprachen von Lexemen

Definition: Grundlegende reguläre Ausdrücke

Grundlegende reguläre Ausdrücke sind

Operator	Ausdruck r	Sprache $L(r)$
Zeichen	`c` aus Σ	$\{\texttt{c}\}$
Stern	`(s)*`	Folge von Wörtern aus $L(\texttt{s})$ sowie λ
Konkatenation	`(st)`	$\{vw \mid v \text{ ist in } L(\texttt{s}) \text{ und } w \text{ ist in } L(\texttt{t})\}$
Alternative	`(s|t)`	$L(\texttt{s})$ vereinigt mit $L(\texttt{t})$

, wobei `s` und `t` jeweils Ausdrücke sind.

Beispiel: Regulärer Ausdruck für TiEL-Bezeichner

- $r =$ `(a|...|z|A|...|Z|0|...|1|_)(a|...|z|A|...|Z|0|...|1|_)*`
- $L(r) = L_{\texttt{IDENTIFIER}}$

Bemerkung

Absteigende Bindungsstärke (alt. Präzedenz) entspricht obiger Reihenfolge; dies ermöglicht das Weglassen von Klammern.

3.4.2 Reguläre Ausdrücke in der Praxis

Erweiterte reguläre Ausdrücke vereinfachen Beschreibung

Definition: Erweiterter regulärer Ausdruck (Regex)

Erweitern grundlegende reguläre Ausdrücke um

Operator	Beispielausdruck r	Sprache $L(r)$
Punkt	`.`	Σ (beliebiges Zeichen)
Zeichenklasse	`[a-cA-c_]`	$\{\texttt{a},\texttt{b},\texttt{c},\texttt{A},\texttt{B},\texttt{C},\texttt{_}\}$
Komplement	`[^ab]`	Beliebiges Zeichen außer a und b
Iteration	`(s+)`	Folge von Wörtern aus $L(\texttt{s})$ ohne λ
Option	`(s?)`	$L(s)$ oder λ

, wobei s ein Regex ist.

Bemerkung

Es gibt verschiedene Regex-Dialekte wie POSIX oder Perl die weitere Operatoren bieten. Mit `\\`, dem Fluchtzeichen (engl. escape symbol), wird die Bedeutung von Sonderzeichen aufgehoben.

Regexe beschreiben TiELs lexikalische Einheiten

Beispiel: Regex der Bezeichner in TiEL

```
[a-zA-Z_]([a-zA-Z_]|[0-9])*
```

Beispiel: Regex der Literale für Zahlen in TiEL

```
[0-9]+(\.[0-9]+)?
```

Beispiel: Regex der Literale für Strings in TiEL

```
"([^"])*"
```

3.5 Lexikalische Analyse mit Scannergeneratoren

3.5.1 Von Ausdrücken zum Scanner

Scannergeneratoren sind Compiler für reguläre Ausdrücke

Definition: »Handgeschriebene« Scanner

`Scanner.java` wird wie »normale« Software entwickelt. Basiert auf Spezifikation von Lexemen für Grundsymbole. Falls ein Lexem gefunden wird, erzeugt der Code Token. Dies ist ein *imperativer* Ansatz: »Wie wird Aufgabe gelöst«.

Definition: Generierter Scanner

`Scanner.java` wird durch einen *Scannergenerator* erzeugt. Lexeme werden für den Generator durch reguläre Ausdrücke *definiert*. Ausdrücke werden mit *Regeln* für die Generierung von Token versehen. Dies ist ein *deklarativer* Ansatz: »Was soll gelöst werden«.

Bemerkung: Vor- und Nachteile der Scannergenerierung

Pro *Generierung beschleunigt Entwicklung und vermeidet Fehler.*
Contra *Generierter Code ist schlechter wartbar und optimierbar.*

3.5.2 Scannergenerator JFlex

JFlex generiert Scannerimplementierung in Java

Scannergenerator: JFlex

JFlex erzeugt Quellcode eines Scanners in Java. JFlex nutzt eigenen Regex-Dialekt. Deklarationen, Regeln und Kommentare ähneln Java.

Ausschnitt aus Beschreibung eines TiEL-Scanners in JFlex

- Definition regulärer Ausdrücke für Lexeme:

```
// aliases
ALPHA = [a-zA-Z_]
DIGIT = [0-9]

// regular definitions

// keywords
FUN = "fun"
// ...
// identifiers
IDENTIFIER = {ALPHA} ({ALPHA} | {DIGIT})*
// literals
NUMBER = {DIGIT}+ ("." {DIGIT}+)?
STRING = \" ([^\"])* \"
// special symbols
LEFT_PAREN = \(
// ...
```

- Regeln mit Java-Code, der Grundsymbol erstellt:

```
// skip whitespace
[\t\n\r ]+   {}
// reserved keywords before identifiers
{FUN}        { return TokenType.FUN; }
// ...
// identifier
{IDENTIFIER} { return TokenType.IDENTIFIER; }
// literals
{NUMBER}     { return TokenType.NUMBER; }
{STRING}     { return TokenType.STRING; }
// special symbols
{LEFT_PAREN} { return TokenType.LEFT_PAREN; }
// ...
```

Generierung eines TiEL-Scanners

```
sudo apt install jflex

jflex tiel.flex

jflex tiel.flex
Reading "tiel.flex"
Constructing NFA : 38 states in NFA
Converting NFA to DFA :
.................
18 states before minimization, 14 states in minimized DFA
Writing code to "Scanner.java"
```

Bemerkung

In Kapitel 4 und 5 beschäftigen wir uns mit der Arbeitsweise von Scannergeneratoren.

3.6 Zusammenfassung

- Die *lexikalische Analyse*, implementiert durch den *Scanner*, verarbeitet schrittweise die Zeichen der Eingabe und generiert *Token* bestehend aus *Tokentyp*, *Lexem* und einem optionalen *Tokenwert*.
- *TiEL-Scanner* ist »handgeschriebener« Scanner auf Basis einer natürlichsprachlichen Beschreibung der Lexeme für Grundsymbole.
- *JFlex* ermöglicht die Generierung eines Scanners mit Hilfe einer deklarativen Programmiersprache, die auf *regulären Ausdrücken* basiert.
- Reguläre Ausdrücke basieren auf *Sprachoperationen*
- *Regex* Syntax und Semantik hängt vom jeweiligen Werkzeug ab.

Literatur

[1] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021. (Kapitel 4)

[2] A. Aho, M. Lam, R. Sethi und J. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2006. (Abschnitt 2.6 und Kapitel 3)

[3] Handpeter Mössenböck. *Compilerbau: Grundlagen und Anwendungen*. dpunkt.verlag, 2024. (Kapitel 2)

[4] Uwe Meyer. *Grundkurs Compilerbau*. Rheinwerk Computing, 2021. (Kapitel 3)

[5] Jan Goyvaerts. Regular-Expression.info, Zugegriffen am 27.04.2025. www.regular-expressions.info/

[6] Till Tantau *Theoretische Informatik*. Vorlesungsskript, WS 2009, Abschnitt 8.3 Online (Kurze Einführung zu regulären Ausdrücken im POSIX-Standard.)

Aufgaben

A 1 (TiEL-Scanner entdecken)

1. Ermitteln Sie die TiEL-Token für den folgenden Programmtext. Was fällt ihnen auf?

```
var a = -12.3;
```

2. Zeichen in der Eingabe werden vom Scanner durch die advance-Methode verarbeitet, in einigen Fällen wird zur Entscheidungsfindung aber auch vorgegriffen und über das aktuelle Zeichen hinaus geschaut ohne es zu verarbeiten. Welche Methoden werden hierfür verwendet und wie weit wird maximal vorgegriffen? Diese Zahl bezeichnet man als den *Lookahead* des Scannern. Je kleiner diese Zahl ist, desto effizienter arbeitet der Scanner auf großen Eingabedateien.
3. TiEL ermöglicht das Ausgaben von Strings mit `print`. Zu welchem Tokentyp führt `print` im TiEL-Programm? Was bedeutet das für die Implementierung der Ausgabefunktion im Interpreter?
4. Wortsymbole (engl. keywords) sind Bezeichner, die in TiEL durch die Sprache reserviert sind. Wie wird im TiEL-Scanner sichergestellt, dass beispielsweise **`while`** in ein Wortsymbol und nicht in einen Bezeichner überführt wird.

A 2 (Lexikalischen Analyse entwickeln)

XML erlaubt die Definition beliebiger *Satzzeichen* (engl. Tags) auf Basis einer Dokumententypdefinition. Wir betrachten das folgende XML-Dokument[1]:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<verzeichnis>
    <titel>Wikipedia Städteverzeichnis</titel>
    <eintrag>
        <stichwort>Genf</stichwort>
        <eintragstext>Genf ist der Sitz von
            ...</eintragstext>
    </eintrag>
    <eintrag>
        <stichwort>Köln</stichwort>
        <eintragstext>Köln ist eine Stadt, die
            ...</eintragstext>
```

[1] Entnommen von de.wikipedia.org/wiki/Extensible_Markup_Language am 23.04.2025 unter CC BY-SA 4.0

```
    </eintrag>
</verzeichnis>
```

Wie lässt sich das obige Dokument in Token (bestehend aus jeweiligem Tokentyp, Lexem und gegebenenfalls Wert) aufteilen? Wie müssen die Token angepasst werden, um zusätzlich Tags mit Attributen wie in `<student givenname="Ada">` zu unterstützen.

Erinnerung: Erweiterte reguläre Ausdrücke. Wir nutzen im Folgenden die Syntax *erweiterter regulärer Ausrücke* (engl. *Regex*) aus der Vorlesung, die folgende Operationen (hier mit einem Beispiel aufgeführt) zur Verfügung stellen: *Stern* wie in `(ab)*`, *Konkatenation* wie in `thm`, *Alternative* wie in `0|1`, *Punkt* wie `.`, *Zeichenklassen* wie `[a-zA-Z]`, *Komplement* wie in `[^ab]`, *Iteration* wie in `(00)+` und *Option* wie in `_?`. Bei der Verwendung der hier aufgeführten *Sonderzeichen* als Zeichen in einem String wird das *Fluchtzeichen* `\` vorangestellt.

A 3 (Reguläre Ausdrücke entwickeln, alte Klausuraufgabe)

Geben Sie reguläre Ausdrücke an für:

1. Wörter beginnend mit einem kleinen `'x'` gefolgt von 2 oder 3 Kleinbuchstaben außer `'x'`.
2. Binärzahlen mit ungerader Anzahl von Ziffern.
3. Einfache URLs, die mit `"http"` oder `"https"` beginnen, gefolgt von einem Doppelpunkt und zwei Schrägstrichen und einem Namen. Ein Name besteht mindestens aus einem und maximal drei Namensteilen, die jeweils durch einen Punkt getrennt sind. Jeder Namensteil besteht aus einem oder mehreren Kleinbuchstaben. Der Name darf nicht mit einem Punkt enden.
   Beispiele: `"https://www"`, `"http://www.thm"` und `"https://server.example.net"`.

A 4 (Reguläre Ausdrücke entwickeln, alte Klausuraufgabe)

Geben Sie zu folgenden Sprachen reguläre Ausdrücke an.

1. Binärzahlen (ohne Vorzeichen) mit durch 3 teilbarer Anzahl von Ziffern.
2. Zuweisungen der Form `"Bezeichner:=Zahl"`, wobei ein Bezeichner mit einem Kleinbuchstaben beginnt und von keinem oder mehreren Klein- oder Großbuchstaben gefolgt ist. Zahl besteht aus einer positiven Anzahl von Ziffern und hat ein optionales Vorzeichen. Vor und nach dem Zuweisungszeichen `":="` darf kein Leerzeichen stehen.
3. Alle Binärzahlen (ohne Vorzeichen), die genau zwei Nullen enthalten und beliebig viele (oder auch keine) Eins.

A 5 (Reguläre Ausdrücke entwickeln, alte Klausuraufgabe)

Geben Sie für die folgenden Eigenschaften von Strings jeweils einen regulären Ausdruck an, der die Sprache aller Strings mit dieser Eigenschaft beschreibt.

1. Passwörter aus Buchstaben (klein und groß) und Ziffern, die mindestens eine Ziffer enthalten.
2. Hexadezimale Zahlen, denen `0x` vorangestellt ist.
3. Dezimalzahlen zwischen 0 und 999, wobei führende Nullen zugelassen sind.

Lösungen

A 1

1. `-` ist nicht Teil der Zahl, sondern ein Operator vor dem Zahl-Literal.
2. Verwendete Methoden sind `peek()` und `match()` mit Lookahead 1 sowie `peekNext()` mit Lookahead 2.
3. `print` ist ein Bezeichner und dessen Verwendung führt zu einem Funktionsaufruf. Diese Funktion ist nicht von den Benutzenden definiert, sondern in den TiEL-Interpreter integriert.
4. Der Scanner erkennt Namen, die zu Bezeichnern und Wortsymbolen führen können, während des Lesens von Zeichen. Abschließend wird dieser Name mit einer vorgegebenen Liste von Wortsymbolen verglichen. Ist der Name in dieser Liste, ist er ein Wortsymbol; ansonsten ist er ein Bezeichner.

A 2

Vergleichen Sie Ihre Lösungsideen mit dem XML-Standard unter www.w3.org/TR/xml11/.

A 3

1. `x[a-wyz][a-wyz][a-wyz]?`
2. `((0|1)((0|1)(0|1))*)`
3. `https?:\/\/[a-z]+(\.[a-z]+(\.[a-z]+)?)?`
   Dieser Ausdruck ohne das Escape-Symbol vor den Zeichen `/` und `.` wird ebenfalls als korrekt gewertet.

A 4

1. `((0|1)(0|1)(0|1))*`
   Lösungen, die alle 8 Möglichkeiten aufzählen, sind ebenfalls richtig.
2. `[a-z][A-Za-z]*:=[+-]?[0-9]+`
3. `1*01*01*`

A 5

1. `[a-zA-Z0-9]*[0-9][a-zA-Z0-9]*`
2. `0x[0-9A-Fa-f]+`
3. `[0-9]?[0-9]?[0-9]`

Kapitel 4

Scannergenerierung: Reguläre Ausdrücke auswerten

4.1 Architektur von Scannergeneratoren 36
4.2 Automaten für reguläre Ausdrücke 37
4.2.1 Nichtdeterministische endliche Automaten 37
4.2.2 Konstruktion von Thompson 38
4.3 Eliminierung von Epsilon-Übergängen 39
4.4 Kontext: Reguläre Ausdrücke für Automaten 40
4.4.1 Transformation am Beispiel 41
4.4.2 ∗ Allgemeine Transformation 42
4.5 Zusammenfassung 43
Aufgaben mit Lösungen 44

Der TiEL-Scanner aus dem vorherigen Kapitel wandelt einen gegebenen Programmtext in eine abstrakte Repräsentation um: Durch Lesen einzelner Zeichen des Quelltextes mit potentieller Vorausschau (engl. look ahead) auf das nächste Zeichen wird jeweils ein Lexem erkannt (engl. recognized) und das zugehörige Grundsymbols (engl. Token) erzeugt. Dieses algorithmische Verfahren ähnelt sich nicht nur bei verschiedenen Compilern und Interpretern für unterschiedliche Programmier- und Beschreibungssprachen, sondern lässt sich durch ein reduziertes Berechnungsmodell der Informatik beschreiben: den endlichen Automaten. Diese Verbindung kann man sich beim Bau von Compilern zu nutzen machen, indem die Lexeme der Grundsymbole durch reguläre Ausdrücke beschrieben werden (letztes Kapitel), in äquivalente nichtdeterministische Automaten überführt werden (dieses Kapitel) und danach simuliert (nächstes Kapitel).

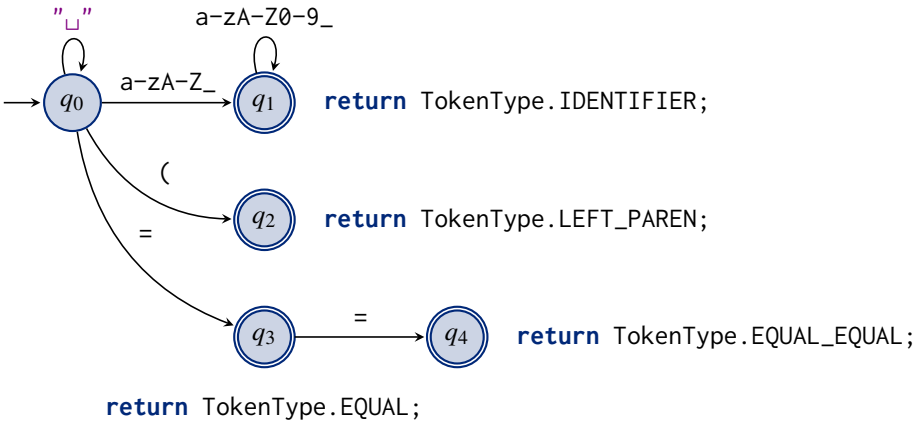
Lernziele

1 *Nichtdeterministische endliche Automaten* (NEAs) und deren Erweiterung um ε-Übergänge kennen und anwenden können.
2 *Thompson-Konstruktion* mit *ε-Eliminierung* zur systematischen Umwandlung regulärer Ausdrücke in NEAs kennen und anwenden können.
3 Umwandlung von NEAs in regulärer Ausdrücke mit Hilfe *generalisierter* NEA*s* kennen und anwenden können.

4.1 Architektur von Scannergeneratoren

Scanner arbeiten konzeptionell wie Automaten für Lexeme

Beispiel: Automat für TiEL-Lexeme mit Regeln (Ausschnitt)



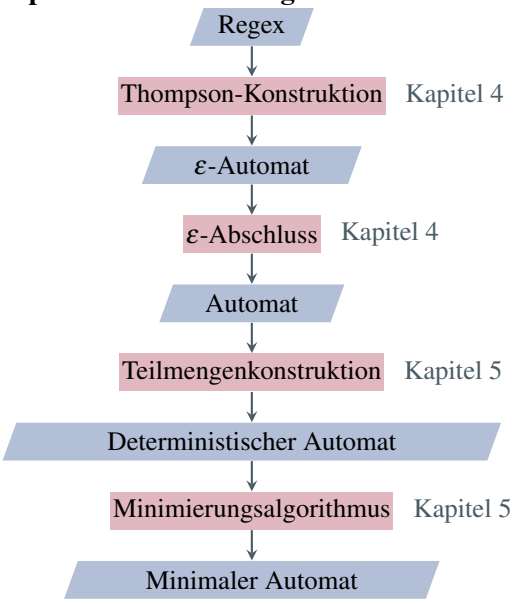
Nächstes Token mit Automaten bestimmen

Der Automat liest, bis kein Übergang mehr möglich ist. Dann wird Token erzeugt beziehungsweise Fehler geworfen.

Dieses Vorgehen entspricht für die regulären Ausdrücke, auf denen der Automat basiert, der *Regel des längsten Musters*: Um das nächste Token zu generieren, suchen wir im aktuell betrachteten Suffix des Eingabeprogramms nach dem längsten Präfix, welches durch einen regulären Ausdruck beschrieben wird. Dies führt über die zugehörige Regel zur Konstruktion eines Tokens. Falls es zwei reguläre Ausdrücke gibt, die einen längstens Präfix beschreiben, wird in JFlex die Regel des Ausdrucks ausgeführt, der weiter oben im Programmtext steht. Dies ist eine Konvention, die man dem hier gezeigten Automaten nicht mehr ans

Scannergeneratoren sind Compiler und arbeiten in Phasen

Pipeline eines Scannergenerators



Bemerkung

Ein Scannergenerator ist ein Compiler für reguläre Ausdrücke. Die Phasen unterscheiden sich je nach Scannergenerator.

4.2 Automaten für reguläre Ausdrücke

4.2.1 Nichtdeterministische endliche Automaten

Endliche Automaten sind »kleine Computer«

Definition: Syntax von NEAs

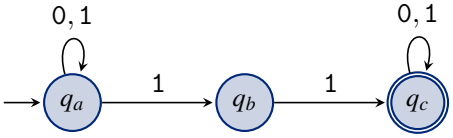
Ein *nichtdeterministischer endlicher Automat* (NEA) ist ein 5-Tupel $A = (\Sigma, Q, q_0, Q_a, \Delta)$ bestehend aus

1. einem *Eingabealphabet* Σ,
2. einer Menge von *Zuständen* Q,
3. einem *Startzustand* q_0,
4. einer Menge *akzeptierender Zustände* $Q_a \subseteq Q$ und
5. einer *Übergangsrelation* $\Delta \subseteq Q \times \Sigma \times Q$.

Beispiel: NEA als Tupel

Sei $A = (\Sigma, Q, q_0, Q_a, \Delta)$ mit $\Sigma = \{0, 1\}$, $Q = \{q_a, q_b, q_c\}$, $q_0 = q_a$, $Q_a = \{q_c\}$ und $\Delta = \{(q_a, 0, q_a), (q_a, 1, q_a), (q_a, 1, q_b), (q_b, 1, q_c), (q_c, 0, q_c), (q_c, 1, q_c)\}$.

Beispiel: NEA als Graph



Nichtdeterminismus bedeutet, dass ein Automat in einem Zustand (hier im Zustand q_a) beim Lesen eines Zeichens (hier das Zeichen 1) unterschiedliche Übergänge wählen kann. Dies entspricht in der Modellierung mit Automaten (und der systematischen Transformation von regulären Ausdrücken zu Automaten) alternativen Wahlmöglichkeiten.

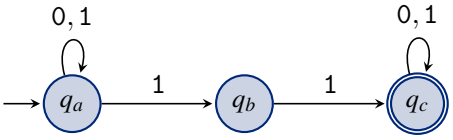
Automaten verarbeiten Eingabe von links nach rechts

Definition: Semantik von NEAs für Worte

Sei $A = (\Sigma, Q, q_0, Q_a, \Delta)$ ein NEA und $w = w_1 \dots w_n$ ein Wort. A *akzeptiert* w, falls es Zustände $r_0, \dots, r_n$ mit folgenden Eigenschaften gibt: $r_0 = q_0$, $(r_i, w_{i+1}, r_{i+1}) \in \Delta$ für jedes $i = 0, \dots, n-1$, und $r_n \in Q_a$. Ansonsten wird w von A *verworfen*.

Definition: Semantik von NEAs für Sprachen

Ein NEA A *akzeptiert* die Sprache $L(A) = \{w \mid A \text{ akzeptiert } w\}$.

Wir betrachten den NEA $A =$ 

Beispiel: Akzeptierte Worte

Es gilt $01110 \in L(A)$, mit Zustandsfolge $q_a, q_a, q_a, q_b, q_c, q_c$ und alternativ $q_a, q_a, q_b, q_c, q_c, q_c$.

Es gilt $010 \notin L(A)$, da keine Zustandsfolge »über q_b hinweg kommt«.

Frage: Akzeptierte Sprache

Welche Sprache akzeptiert A?

ε-Übergänge verarbeiten kein Zeichen der Eingabe

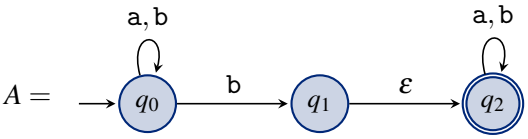
Definition: Syntax von ε-NEAs

Ein ε-NEA ist ein NEA, mit erweiterter *Übergangsrelation* $\Delta \subseteq Q \times \Sigma \cup \{\varepsilon\} \times Q$, wobei ε nicht in Σ ist.

Definition: Semantik von ε-Übergängen

Für Übergänge mit ε (sprich »epsilon«) wird kein Zeichen gelesen.

Beispiel



`abaa` $\in L(A)$ und `aaa` $\notin L(A)$ sowie $L(A)$ = Wörter mit a und b, die mindestens ein b enthalten.

Bemerkung

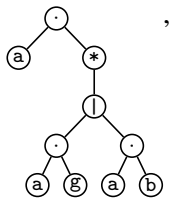
ε ist kein Zeichen oder String, sondern ein Steuersignal.

Alternativ wird häufig ε (alternativ zu oder anstatt λ) als Zeichen für den leeren String verwendet. Dann kann ein ε-Übergang als ein Übergang betrachtet werden, den man mit einem leeren String wählen kann.

4.2.2 Konstruktion von Thompson

Reguläre Ausdrücke sind hierarchisch aufgebaut

Beispiel: Regulärer Ausdruck als Baum

- Regulärer Ausdruck $r =$ `a((ag|ab)*)`
- Baum $T =$  , wobei $\odot$ für die Konkatenation steht.

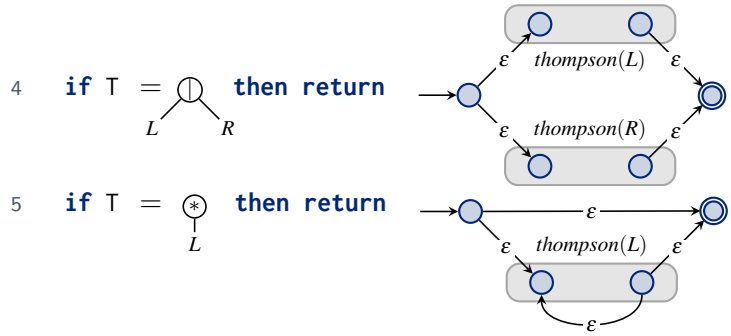
Algorithmus: Automaten für Ausdrücke (Idee)

Erstelle Automaten für die Blätter des Baums. Erstelle von unten nach oben Automaten für die inneren Knoten. Wähle eine knappe Implementierung als rekursiven Algorithmus.

Ausdruck wird rekursiv in Automat transformiert

Algorithmus: Thompson-Konstruktion für grundlegende Ausdrücke

```
1 Automaton method thompson(Regex r viewed as a tree T)
2   if T = σ with σ ∈ Σ return  →○—σ→◎
3   if T = ·  then return  →○ thompson(L) ○ thompson(R) ◎
          L   R
```

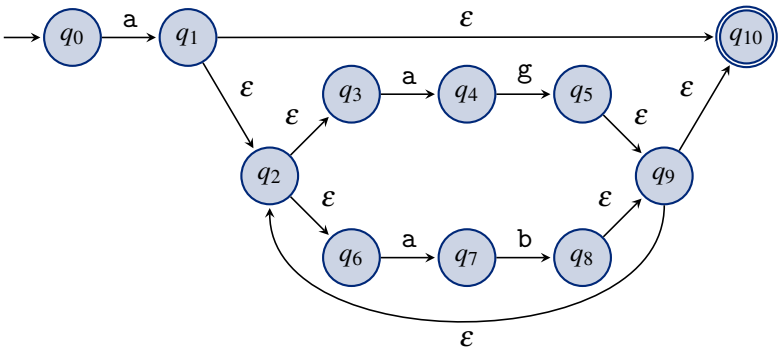


Automaten werden im Algorithmus durch Hinzufügen von Zuständen und Übergängen kombiniert. Dabei werden initiale und akzeptierende Zustände aktualisiert; gegebenenfalls identifiziert.

Aussage
Es gilt $L(r) = L(A)$ *mit* $A =$ `thompson(r)`.

Thompson-Konstruktion erzeugt viele ε-Übergänge

Beispiel: Thompson-Automat für `a((ag|ab)*)`



Aussage
Der Thompson-Automat A hat maximal 2 mal so viele Zustände wie Operationen im Ausdruck sind und genau einen akzeptierenden Zustand ohne Übergänge.

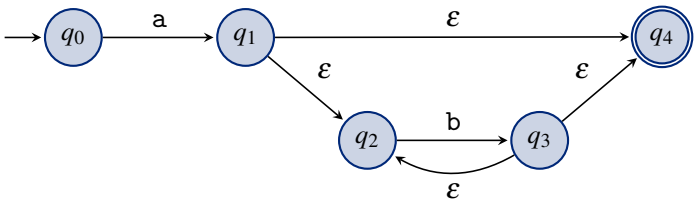
4.3 Eliminierung von Epsilon-Übergängen

ε-Abschluss überspringt ε-Übergänge

Definition: ε-Abschluss
Für einen NEA A und Zustand q enthält die Menge ε-closure(A,q) alle Zustände, die durch ε-Übergänge von q in A erreichbar sind.

Beispiel

- Thompson-Automat A für `ab*`



- ε-closure$(A, q_3) = \{q_3, q_2, q_4\}$

Bemerkung

Der Begriff Abschluss bedeutet, dass man den ε-Übergängen abschließend (»bis zum Ende«) nachgeht.

ε-Übergänge werden durch σ-Übergänge ersetzt

Algorithmus: Elimination der ε-Übergänge

```
1  Automaton method eliminate(Automaton A)
2  A' ← copy A
3  foreach Zustand q in A do
4    foreach p ∈ ε-closure(A, q) do
5      if p is accepting in A then
6        make q accepting in A'
7      foreach transition (p, σ, r) with σ ≠ ε in A do
8        insert transition (q, σ, r) into A'
9  eliminate all ε-transitions from A'
10 eliminate all states from A' unreachable from start state
11 return A'
```

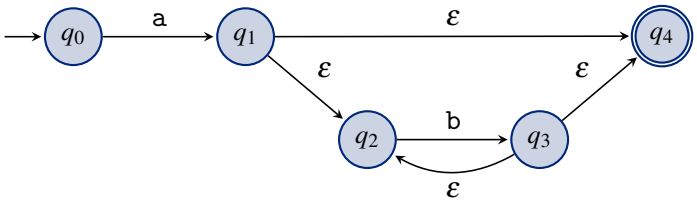
Aussage

$L(A) = L(\texttt{eliminate}(A))$

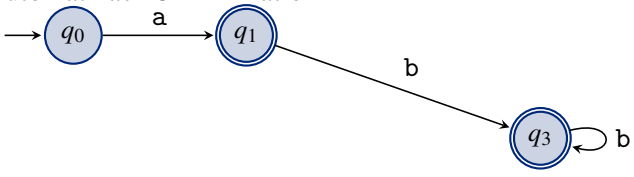
Ohne ε-Übergänge haben Zustände viele Ausgänge

Beispiel: nach ε-Elimination

- Thompson-Automat A für `ab*`



- Automat nach ε-Elimination



4.4 Kontext: Reguläre Ausdrücke für Automaten

Automaten haben keine hierarchische Struktur

Folgerung

Für jeden regulären Ausdruck r gibt es einen NEA *A mit $L(r) = L(A)$.*

Lemma

Für jeden NEA *A gibt es einen regulären Ausdruck r mit $L(A) = L(r)$.*

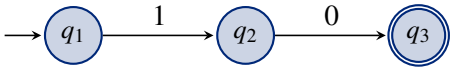
Beweis wandelt Automat iterativ in Ausdruck um. Reguläre Ausdrücke haben eine *hierarchische Struktur*, was deren Umwandlung in NEAs ermöglicht. Automaten haben *keine hierarchische Struktur*. Idee: Wir beschreiben Pfade über Zustände mit Ausdrücken, schreiben die Ausdrücke an den Automaten und lassen den Zustand weg.

4.4.1 Transformation am Beispiel

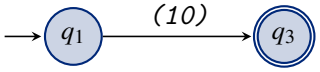
Idee bei sequenzieller Komposition

Beispiel

- NEA A vor der Transformation, aus dem wir q_2 löschen:



- Der transformierte *generalisierte Automat* G hat zum Ausgleich einen *Übergang mit regulärem Ausdruck*:

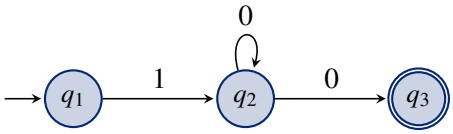


Bemerkung
G liest mit einem Übergang einen Teilstring anstatt ein Symbol. Es gilt $L(A) = L(G)$. Für den regulären Ausdruck r auf der Kante von G gilt $L(A) = L(r)$.

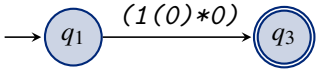
Idee bei Zustand mit Wiederholungen

Beispiel

- Der NEA hat eine Schlaufe am Zustand q_2, den wir löschen:



- Wir »heilen« verlorene Pfade über q_2, indem wir einen *Übergang mit regulärem Ausdruck* verwenden:

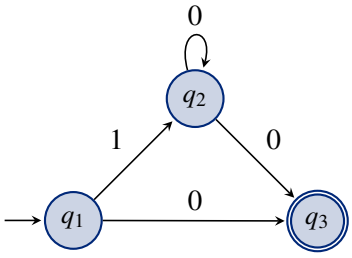


Bemerkung
Der generalisierte Automat akzeptiert wieder die gleiche Sprache. Der reguläre Ausdruck beschreibt wieder die Sprache des NEAs.

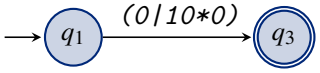
Idee bei paralleler Komposition

Beispiel

- Der NEA hat parallele Pfade und wir löschen wieder q_2:



- Ausgleich des gelöschten Pfades über q_2 durch Ausdruck:



Bemerkung

Der generalisierte Automat ermöglicht beide Pfade auf einer Kante. Der reguläre Ausdruck beschreibt wieder die ursprüngliche Sprache.

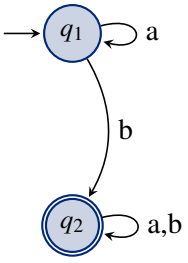
4.4.2 ∗ Allgemeine Transformation

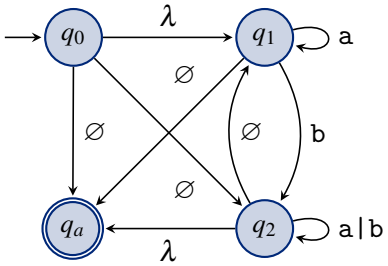
Schritt 1: Umwandlung von NEA in generalisierten NEAs

Definition: Syntax und Semantik generalisierter NEAs

Ein generalisierter NEA (GNEA) G ist ein NEA, aber *Übergänge haben reguläre Ausdrücke* als Bedingung, der *Startzustand* hat keine eingehenden Kanten, es gibt genau einen *akzeptierenden Zustand* ohne ausgehende Kanten, und ansonsten haben *alle Zustandspaare Kanten*, wobei $\varnothing$ der reguläre Ausdruck mit $L(\varnothing) = \emptyset$ ist.

Umwandlung von NEA in äquivalenten GNEA am Beispiel





Schritt 2: Iterativ Zustand löschen und Automat »heilen«

Operation: löschen-und-heilen

Wir löschen einen Zustand q aus G, der nicht Start oder akzeptierend ist. Jeder Übergang von G mit Ausdruck r von q_i nach q_j bekommt den Ausdruck

$$(r_1)(r_2)*(r_3)\,|\,(r)\,, \textit{wobei}$$

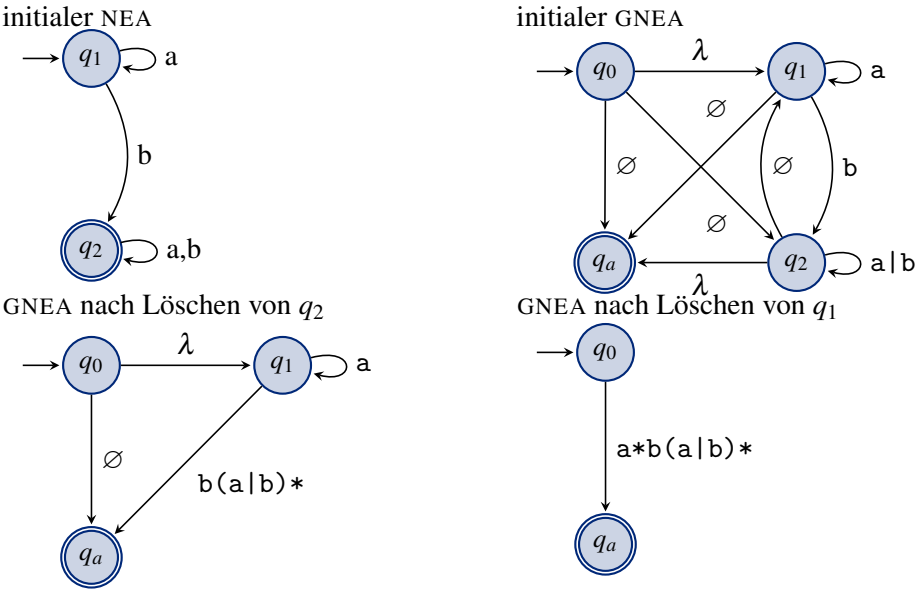
- r_1 der Übergang von q_i nach q ist,
- r_2 der Übergang von q nach q ist und
- r_3 der Übergang von q nach q_j ist.

Lemma

Sei G ein GNEA und G' der GNEA nach einer Anwendung von löschen-und-heilen. Dann gilt $L(G) = L(G')$.

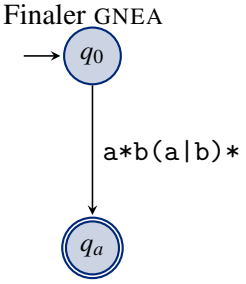
Begründung. Abläufe in G leiten wir um, wenn sie q enthalten. Durch die Konstruktion entstehen keine neuen Abläufe.

Iteratives löschen-und-heilen



Schritt 3: Regulären Ausdruck ablesen

Ablesen der Lösung vom einzigen Übergang



Regulärer Ausdruck als Ergebnis
`a*b(a|b)*`

Bemerkung

Die Korrektheit der Transformation folgt aus der Umwandlung von einem NEA *zu einem äquivalenten* GNEA*, der mehrfachen Umwandlung von* GNEA*s zu kleinen äquivalenten* GNEA*s, dem Ablesen des Ausdrucks auf der einzigen Kante.*

4.5 Zusammenfassung

- Die Arbeitsweise von Scannern lässt sich durch Automaten repräsentieren.
- *Scannergeneratoren* überführen reguläre Ausdrücke in implementierte Automaten; sie sind »Compiler für reguläre Ausdrücke« und arbeiten in Phase.
- Die *Thompson-Konstruktion* wandelt reguläre Ausdrücke in ε-NEA*s* um.
- ε-NEAs können mit Hilfe der des ε-*Abschlusses* in NEAs ohne ε-Übergänge umgewandelt werden.
- Automaten lassen sich (prinzipiell) auch in reguläre Ausdrücke mit Hilfe *generalisierter* NEA*s* umwandeln; dies findet keine Anwendung in Scannergeneratoren.

Literatur

[1] A. Aho, M. Lam, R. Sethi und J. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2006. (Abschnitte 3.4 bis 3.11)

[2] Uwe Meyer. *Grundkurs Compilerbau*. Rheinwerk Computing, 2021. (Abschnitt 3.4)

[3] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 2013, Abschnitt 1.3. (Reguläre Ausdrücke für Automaten.)

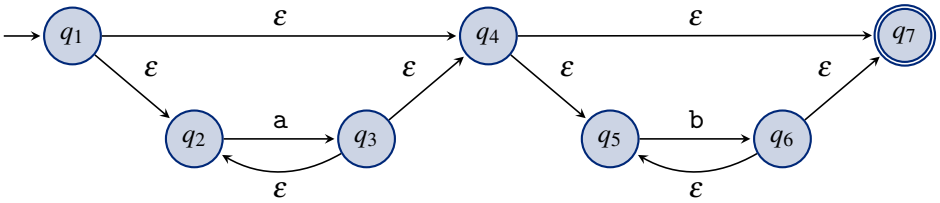
Aufgaben

A 1 (Regulären Ausdruck in ε-NEA überführen)

Wenden Sie die Thompson-Konstruktion an, um den regulären Ausdruck `(a|b)*abb` in einen gleichbedeutenden ε-NEA A zu überführen.

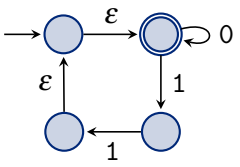
A 2 (ε-Übergänge eliminieren)

Wenden Sie das systematische Verfahren aus der Vorlesung an, um die ε-Übergänge im folgenden Automaten zu eliminieren:



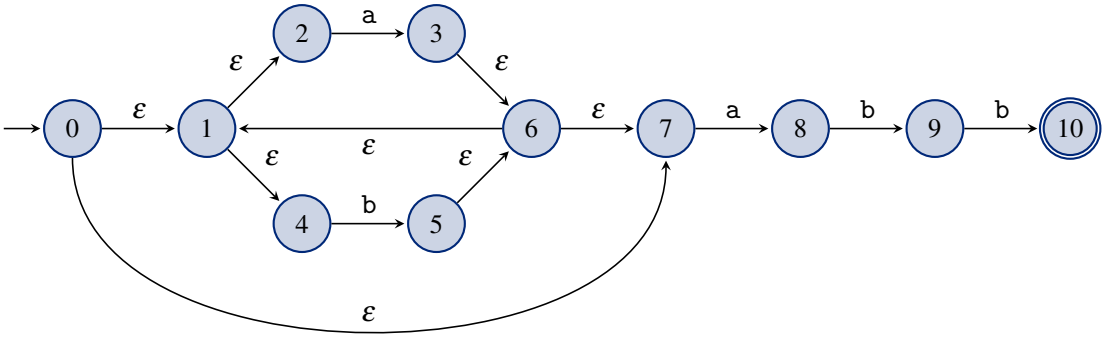
A 3 (Regulären Ausdruck in NEA überführen, alte Klausuraufgabe)

1. Wenden Sie die Thompson-Konstruktion an, um den regulären Ausdruck `(0|1)*2` in einen gleichbedeutenden ε-NEA zu überführen.
2. Wandeln Sie den folgenden ε-NEA in einen gleichbedeutenden NEA (ohne ε-Übergänge) um:

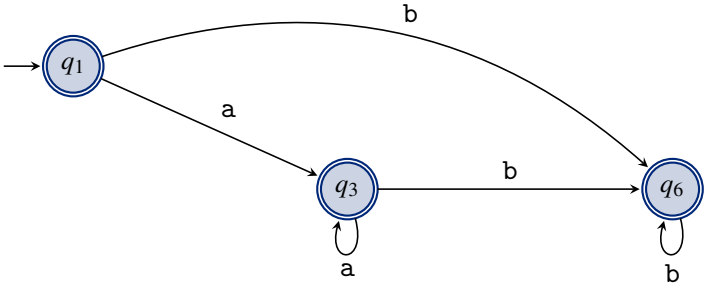


Lösungen

A 1

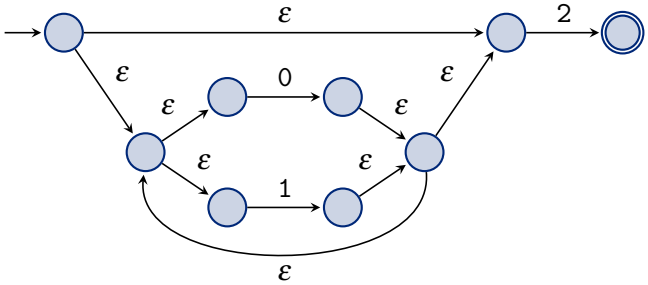


A 2

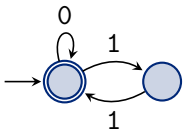


A 3

1.



2.



Kapitel 5

Scannergenerierung: Automaten simulieren

5.1 Motivation 47
5.2 Von Nichtdeterminismus zu Determinismus 47
5.2.1 Deterministische endliche Automaten 48
5.2.2 Teilmengenkonstruktion 49
5.3 Minimierung deterministischer Automaten 51
5.3.1 Zustandsäquivalenz 51
5.3.2 Minimierungsalgorithmus 52
5.4 Zusammenfassung 53
Aufgaben mit Lösungen 53

Im vorherigen Kapitel haben wir ein algorithmische Verfahren kennen gelernt, welches reguläre Ausdrücke in gleichbedeutende Automaten überführt. In diesen Automaten wurde ε-Übergänge, die durch die Thompson-Konstruktion entstehen können, entfernt, sie können aber weiterhin nichtdeterministisch sein. Dies führt zu folgender Situation: Entweder wir handhaben den Nichtdeterminismus während der Simulation des Automaten. Dies würde bedeutet, dass wir verschiedenen Ausführungspfaden folgen, um festzustellen, ob einer der Pfade akzeptierend ist. Alternativ können wir den nichtdeterministischen Automaten zunächst in einen äquivalenten deterministischen Automaten umwandeln, um danach eine vereinfachte Simulation zu nutzen, die nur einem Ausführungspfad folgt. Für Compiler und Interpreter, die sich nicht dynamisch ändern, sondern höchstens mit der nächsten Programmiersprachenversion, eignet sich die zusätzliche Umwandlung, die wir in diesem Kapitel betrachten. Dies ermöglicht es uns zudem die Größe des Automaten nach der Umwandlung zu minimieren; dies verringert die Größe der Beschreibung des Scanners.

Lernziele

1 Nichtdeterministische von *deterministischen endlichen Automaten* (DEAs) abgrenzen können.
2 *Teilmengenkonstruktion* anwenden können.
3 *Minimierungsalgorithmus* anwenden können.

5.1 Motivation

Ziel dieses Kapitels: Effiziente Simulation des Automaten

Erreichtes aus vorherigen Kapiteln

- Lexeme der Grundsymbole durch reguläre Ausdrücke beschreiben.
- Reguläre Ausdrücke in NEAs umgewandelt.

Ziel in diesem Kapitel

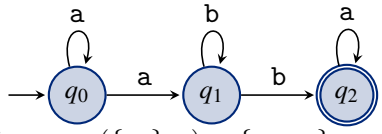
- *Erzeugten Automaten auf Eingabe eines Strings x simulieren.*
- *Ziel 1: Laufzeit linear in Eingabegröße; das heißt $O(|x|)$.*
- *Ziel 2: Automat verkleinern, um Programmgröße zu reduzieren.*

5.2 Von Nichtdeterminismus zu Determinismus

Dauer der Simulation eines NEA hängt von Automat ab

Definition: Schritt eines NEA
Sei $A = (\Sigma, Q, q_0, Q_a, \Delta)$ ein NEA. Für eine Menge von Zuständen $R \subseteq Q$ und $\sigma \in \Sigma$ ist $\text{move}(R, \sigma) := \{q \text{ aus } Q \mid \text{es gibt } r \text{ in } R \text{ mit } (r, \sigma, q) \text{ in } \Delta\}$.

Beispiel

- $A =$ 

- Schritt: $\text{move}(\{q_0\}, a) = \{q_0, q_1\}$
- Schritt: $\text{move}(\{q_0, q_1\}, b) = \{q_1, q_2\}$

Algorithmus: Simulation von NEAs

```
1 bool simulate(NEA A = (Σ, Q, q0, Qa, Δ), String x)
2   R ← {q0}
3   while x has next character do
4     σ ← nextCharacter(x)
5     R ← move(R, σ)
6   return R ∩ Qa ≠ ∅
```

Beispiel
$x =$ `aab`; $\text{move}(\{q_0\}, a) = \{q_0, q_1\}$, $\text{move}(\{q_0, q_1\}, a) = \{q_0, q_1\}$, $\text{move}(\{q_0, q_1\}, b) = \{q_1, q_2\}$.

Aussage
`simulate` *kann mit einer Laufzeit in* $O(|x| \cdot |Q|^2)$ *implementiert werden.*

Begründung. Jedes der $|x|$ Zeichen der Eingabe wird genau einmal gelesen. R enthält maximal $|Q|$ Zustände aus denen für jedes Zeichen maximal $|Q|$ Übergänge herausführen. Die Methode kann mit geeigneten Datenstrukturen für die Repräsentation des Automaten und die Aktualisierung effizienter implementiert werden (siehe Literatur), allerdings ist die Laufzeit weiterhin abhängig von der Größe des Automaten und nicht nur der Eingabe.

Bemerkung

Für Lexemerkennung (Methode `nextToken` *im TiEL-Interpreter) verlässt man die Methode vorzeitig, wenn R als nächstes $\emptyset$ wäre. Zudem wir die Simulation nach der letzten verarbeiteten Position im Quellprogramm x neu angestoßen.*

5.2.1 Deterministische endliche Automaten

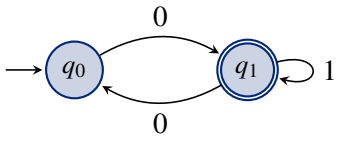
Ohne Nichtdeterminismus sind Übergänge eindeutig

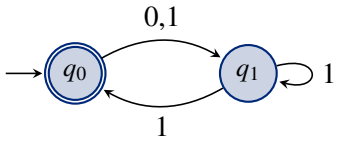
Definition: Aufbau eines DEAs

Ein *deterministischer endlicher Automat (*DEA*)* ist ein NEA $A = (\Sigma, Q, q_0, Q_a, \Delta)$ mit folgender Eigenschaft: Für jeden Zustand $r \in Q$ und Zeichen $\sigma \in \Sigma$ gibt es maximal einen Zustand q mit $(r, \sigma, q) \in \Delta$.

Frage

Welcher der folgenden Automaten ist ein DEA, welcher nicht?

- $A =$ 

- $B =$ 

DEAs haben für jede Eingabe genau einen Ablauf

Bemerkung

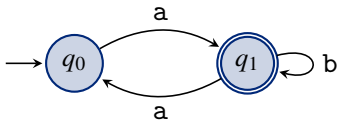
DEA*s sind eingeschränkte* NEA*s; somit überträgt sich die Semantik. Ein* DEA *akzeptiert mit maximal einer möglichen Zustandsfolge.*

Aussage

Für DEA*s hat* `simulate` *eine Laufzeit von $O(|x|)$.*

Begründung. Die Menge der aktuellen Zustände R enthält maximal einen Zustand r. r besitzt für das nächste Symbole σ maximal einen Übergang.

Beispiel

- $A =$ 

- $x =$ `aaab`; move$(\{q_0\}, a) = \{q_1\}$, move$(\{q_1\}, a) = \{q_0\}$, move$(\{q_0\}, a) = \{q_1\}$, move$(\{q_1\}, b) = \{q_1\}$.

5.2.2 Teilmengenkonstruktion

Zustandsmengen sind die Zustände eines äquivalenten DEAs

Idee: Simulation von NEA durch DEA

Der NEA ist zu jedem Zeitpunkt in einer Teilmenge von Zuständen. DEA simuliert exakt diese Folge von Zustandsmengen.

Definition: Teilmengenautomat

Sei $A = (\Sigma, Q, q_0, Q_a, \Delta)$ in NEA. Der *Teilmengenautomat* von A ist ein DEA $B = (\Sigma, Q', q_0', Q_a', \Delta')$ mit

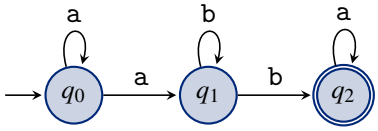
- Σ ist das unveränderte Eingabealphabet,
- $Q' = \{R \mid R \subseteq Q\}$; Teilmengen als Zustände.
- $q_0' = \{q_0\}$
- $Q_a' = \{R \in Q' \mid R \cap Q_a \neq \emptyset\}$
- Für jede Menge $R \in Q'$ und Zeichen $\sigma \in \Sigma$ ist $(R, \sigma, \text{move}(R, \sigma))$ in Δ'.

Aussage

Für jeden NEA A und Teilmengenautomat B gilt $L(A) = L(B)$.

Teilmengenautomat wird mit Hilfe einer Tabelle konstruiert

Beispiel

Wir betrachten NEA $A =$ 

.

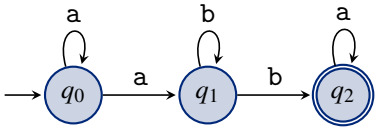
Teilmengenautomat B als 5-Tupel ist $B = (\Sigma, Q', q_0', Q_a', \Delta')$ mit

- $\Sigma = \{a, b\}$,
- $Q' = \{\emptyset, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \{q_1, q_2\}, \{q_0, q_2\}, \{q_0, q_1, q_2\}\}$
- $q_0' = \{q_0\}$
- $Q_a' = \{\{q_2\}, \{q_1, q_2\}, \{q_0, q_2\}, \{q_0, q_1, q_2\}\}$
- Δ ist definiert durch

Zustand R	move$(R, \mathtt{a})$	move$(R, \mathtt{b})$
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0\}$	$\{q_0, q_1\}$	$\emptyset$
$\{q_1\}$	$\emptyset$	$\{q_1, q_2\}$
$\{q_2\}$	$\{q_2\}$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\{q_2\}$	$\{q_1, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\emptyset$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$

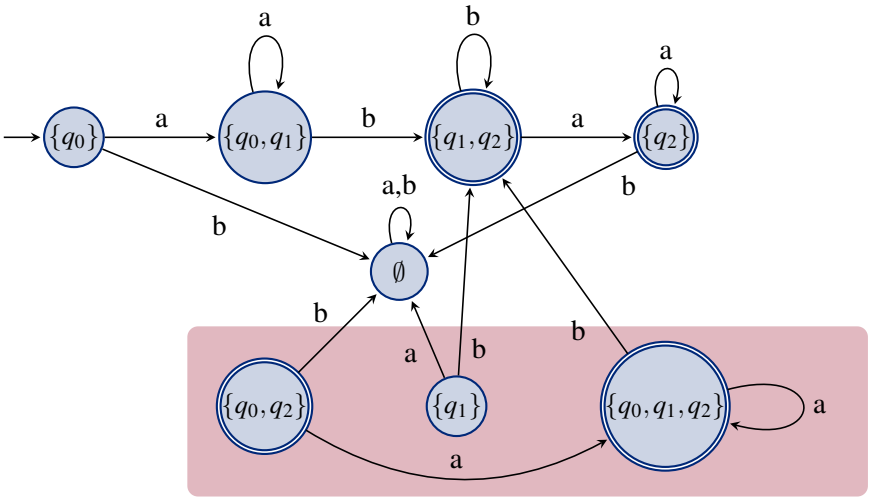
Der Teilmengenautomat kann ebenfalls visualisiert werden

Beispiel

Wir betrachten weiterhin den NEA $A =$ 

.

Teilmengenautomat B als Graph inklusive *nicht-erreichbarer Zustände*:



Scannergeneratoren bauen Teilmengenautomat schrittweise

Bemerkung

Teilmengenautomat hat Zustände, die für keine Eingabe erreichbar sind. Wir konstruieren erreichbare Zustände ähnlich der NEA-Simulation.

Algorithmus

```
1 DEA method subset(NEA A = (Σ, Q, q0, Qa, Δ))
2   B ← (Σ, Q', {q0}, Q'a, Δ') with Q' = {q0}, Q'a = ∅, Δ = ∅
3   while there is R ∈ Q' without σ − transition do
4     P ← move(R, σ)
5     Q' ← Q' ∪ {P}
6     if P ∩ Qa ≠ ∅ then Qa ← Qa ∪ {P}
7     Δ' ← Δ' ∪ {(R, σ, P)}
8   return B
```

Beispiel

Δ ist definiert durch

Zustand R	move$(R,\mathtt{a})$	move$(R,\mathtt{b})$
$\{q_0\}$	$\{q_0,q_1\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0,q_1\}$	$\{q_0,q_1\}$	$\{q_1,q_2\}$
$\{q_1,q_2\}$	$\{q_2\}$	$\{q_1,q_2\}$
$\{q_2\}$	$\{q_2\}$	$\emptyset$

Aussage

Für jeden NEA A *gilt* $L(A) = L(B)$ *mit* B = `subset(`A`)`.

Teilmengenkonstruktion kann große Automaten erzeugen

Bemerkung

Durch iterative Konstruktion werden nur erreichbare Zustände konstruiert. Trotzdem kann die Konstruktion notwendigerweise sehr groß werden.

Beispiel

- Sei $\Sigma = \{0,1\}$ und für jedes $k \in \mathbb{N}$ bezeichne L_k die Sprache aller Wörter über Σ, deren k-letztes Symbol eine 0 ist.
- Für $k = 3$ gilt $\texttt{1011} \in L_3$, aber $\texttt{0100} \notin L_3$.
- Für jedes $k \in \mathbb{N}$ gibt es einen NEA A_k mit $k+1$ Zuständen und $L(A_k) = L_k$.
- Für jedes $k \in \mathbb{N}$ hat ein DEA B_k mit $L(B_k) = L_k$ mindestens $2^k - 1$ Zustände.

Folgerung

Es gibt NEAs *für die die Teilmengenkonstruktion einen exponentiell größeren* DEA *erzeugt.*

Folgerung

Es gibt NEAs*, deren* DEAs *sich nach der Teilmengenkonstruktion nicht verkleinern lassen.*

Bemerkung

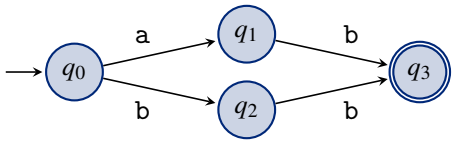
Für reguläre Ausdrücke und zugehörige NEAs *aus der lexikalischen Analyse werden* DEAs *in der Regel nicht exponentiell große. Zudem lassen Sie sich häufig weiter (systematisch) verkleinern.*

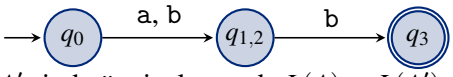
5.3 Minimierung deterministischer Automaten

5.3.1 Zustandsäquivalenz

Äquivalente Zustände können zusammengefasst werden

Beispiel: Großer und verkleinerter DEA

- $A =$ 

mit »äquivalenten« q_1 und q_2.
- $A' =$ 

mit »verschmolzenem« $q_{1,2}$.
- A und A' sind »äquivalent«, da $L(A) = L(A') = \{\texttt{ab}, \texttt{bb}\}$.

Definition

Zwei NEAs A und A' sind *äquivalent*, falls $L(A) = L(A')$.

Definition: Äquivalenz von Zuständen

Sei $A = (\Sigma, Q, q_0, Q_a, \Delta)$ ein DEA. Zwei Zustände $q, r \in Q$ sind *äquivalent*, falls für jeden String x gilt: x wird genau dann ausgehend von q akzeptiert, wenn x ausgehend von r akzeptiert wird.

Aussage

Falls q, r nicht äquivalent sind, dann mit einem x der Länge maximal $|Q|$.

Bemerkung

Siehe die Literatur am Ende dieses Kapitels für einen Beweis dieser Aussage.

5.3.2 Minimierungsalgorithmus

Minimierung markiert nicht äquivalente Zustände iterativ

Definition

Δ ist *total*, falls es einen Übergang für jeden Zustand und jedes Symbol gibt.

Algorithmus

```
1  method minimize(DEA A = (Σ,Q,q0,Qa,Δ)) // A has total transitions
2    T ← 2−dimensional table indexes by states
3    foreach pair r,q ∈ Q with r ∈ Qa and q ∉ Qa do
4      mark T(r,q) // mark as not equivalent
5    do
6      foreach unmarked pair r,q ∈ Q, σ ∈ Σ do
7        lookup (r,σ,r') ∈ Δ, (q,σ,q') ∈ Δ
8        if T(r',q') is marked then
9          mark T(r,q)
10   while T is modified
11   iterative merge unmarked pairs
```

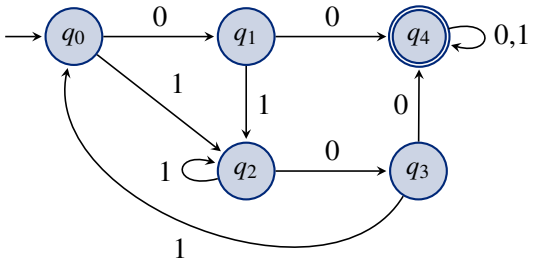
Aussage

`minimize` *erzeugt äquivalente* DEA*s für* DEA*s mit totaler Übergangsrelation.*

Symmetrische Tabelle wird als »Dreieck« dargestellt

Beispiel

- Eingabe:



- Tabelle der Zustandspaare:

q_1				
q_2				
q_3				
q_4				
	q_0	q_1	q_2	q_3

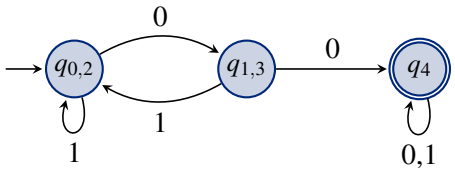
- Initiales markieren:

q_1				
q_2				
q_3				
q_4	$\star$	$\star$	$\star$	$\star$
	q_0	q_1	q_2	q_3

- Wiederholtes markieren:

q_1	★			
q_2		★		
q_3	★		★	
q_4	★	★	★	★
	q_0	q_1	q_2	q_3

- Ausgabe:



5.4 Zusammenfassung

- *Scannergeneratoren* sind Compiler für reguläre Ausdrücke
- Die *Teilmengenkonstruktion* wandelt einen NEA in einen DEA um und ermöglicht so eine schnellere Lexemerkennung; auf Kosten einer langsameren Scannergenerierung.
- Der *Minimierungsalgorithmus* ermöglicht durch einen kleineren Automaten die Generierung von weniger Programmcode für den Scanner.
- Generierte Scanner simulieren einen Automaten entweder explizit mit Hilfe einer Übergangstabelle oder implizit auf Basis generierter bedingter Anweisungen.

Literatur

[1] A. Aho, M. Lam, R. Sethi und J. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2006. (Abschnitte 3.4 bis 3.11)

[2] Uwe Meyer. *Grundkurs Compilerbau*. Rheinwerk Computing, 2021. (Abschnitt 3.4)

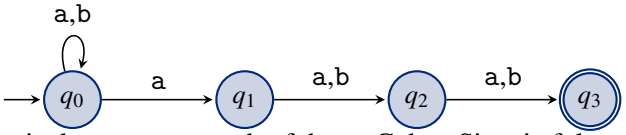
[3] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 2013, Abschnitt 1.2. (Teilmengenkonstruktion.)

[4] Uwe Schöning. *Theoretische Informatik - kurz gefasst*. Spektrum Akademischer Verlag, 2008, Abschnitt 1.2.2 und 1.2.5 (Teilmengenkonstruktion und Minimierungsalgorithmus)

Aufgaben

A 1 (Systematische Überführung von NEAs in DEAs)

Wenden Sie die Teilmengenkonstruktion an, um den NEA $A =$



mit Alphabet $\Sigma = \{\mathtt{a}, \mathtt{b}\}$ in einen äquivalenten DEA zu überführen. Gehen Sie wie folgt vor:

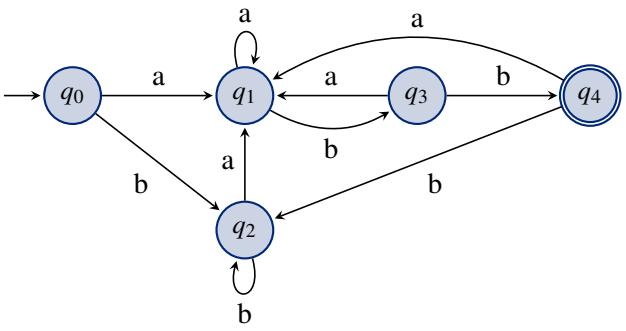
1. Simulieren Sie das Verhalten des Algorithmus `subset` aus dem Skript, indem Sie die folgende Tabelle übernehmen und erweitern:

Zustand R	move$(R,\mathtt{a})$	move$(R,\mathtt{b})$
$\{q_0\}$	$\{q_0,q_1\}$	$\{q_0\}$
	...	

2. Zeichnen Sie den resultierenden DEA. Nutzen Sie Mengen von Zuständen als Namen für Zustände.

A 2 (Systematische Minimierung von DEAs)

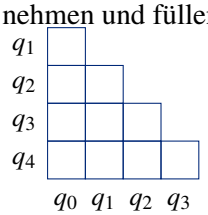
Wenden Sie den Minimierungsalgorithmus an, um die Anzahl der Zustände des DEA



$A =$

mit totaler Übergangsrelation zu minimieren. Gehen Sie dabei wie folgt vor:

1. Simulieren Sie das Verhalten des Algorithmus, indem Sie die folgende Tabelle übernehmen und füllen:

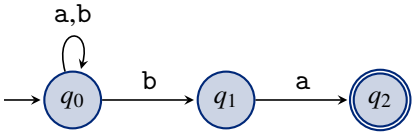


2. Geben Sie den resultierenden minimalen DEA A' in graphischer Repräsentation an. Nutzen Sie für die Zustände die Namen, die sich durch das verschmelzen von Zuständen ergeben.
3. Welcher möglichst kurze reguläre Ausdruck beschreibt die von A' erkannte Sprache.

A 3 (DEA für Regex konstruieren)

Konstruieren Sie für den erweiterten regulären Ausdruck $r =$ `[0-9]+(\.[0-9]+)?`, der die in TiEL zulässigen Zahlenliterale beschreibt, einen äquivalenten DEA A; das heißt, es soll $L(r) = L(A)$ gelten. Geben Sie den Automaten in graphischer Repräsentation an. (Hinweis: Die Konstruktion muss nicht mit den systematischen Verfahren aus dem Skript durchgeführt werden. Wenn Sie sich jedoch für die systematischen Verfahren entscheiden, erweitern Sie zunächst die Thompson-Konstruktion um die Operatoren + und ?.

A 4 (NEA in DEA überführen, alte Klausuraufgabe)

Wenden Sie den Teilmengenalgorithmus an, um den NEA 

mit Alphabet $\Sigma = \{\mathtt{a},\mathtt{b}\}$ in einen äquivalenten DEA zu überführen. Gehen Sie dabei wie folgt vor.

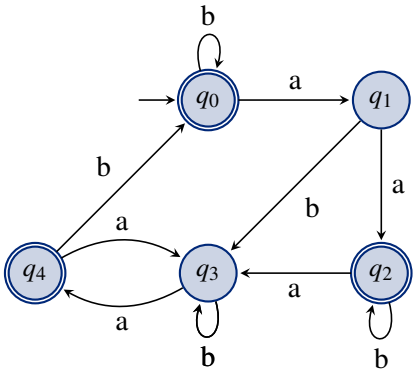
1. Simulieren Sie das Verhalten des Algorithmus, indem Sie die folgende Tabelle übernehmen, füllen und um Zeilen erweitern:

Zustand R	move$(R, \mathtt{a})$	move$(R, \mathtt{b})$
$\{q_0\}$		

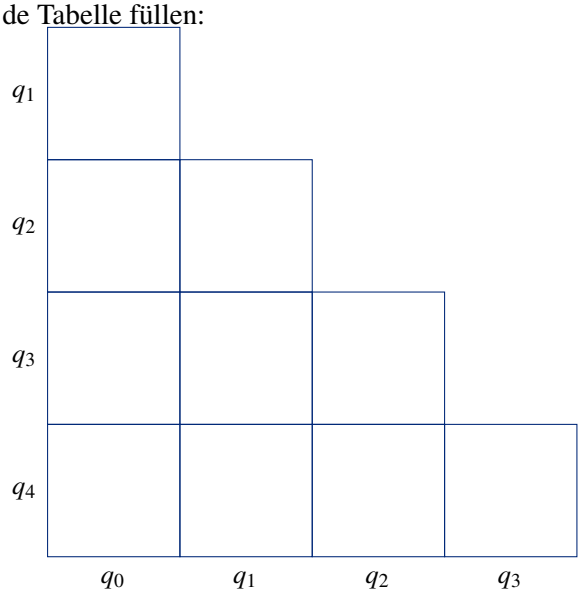
2. Geben Sie den resultierenden DEA in Graphdarstellung an.

A 5 (DEA minimieren, alte Klausuraufgabe)

Wir betrachten den folgenden DEA:



1. Wenden Sie den Minimierungsalgorithmus an, um die Anzahl der Zustände des DEA zu minimieren. Simulieren Sie das Verhalten des Algorithmus, indem Sie die folgende Tabelle füllen:



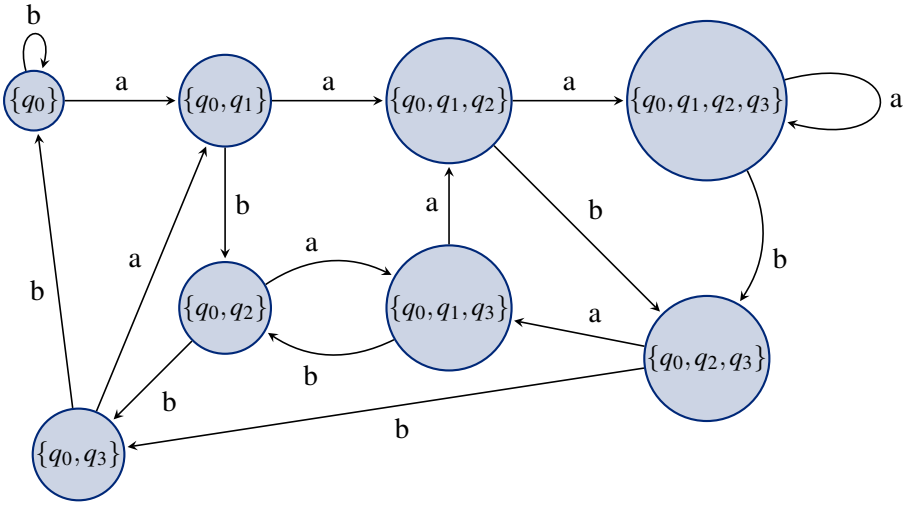
2. Geben Sie den resultierenden DEA an.

Lösungen

A 1

1. Gefüllte Tabelle:

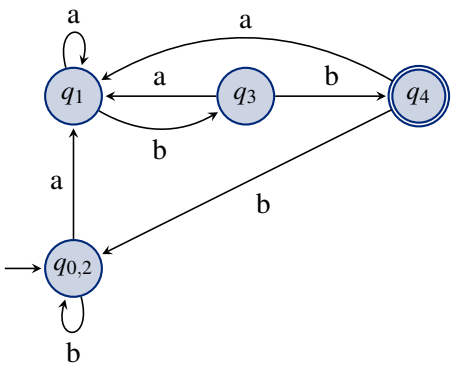
Zustand R	move$(R,\mathtt{a})$	move$(R,\mathtt{b})$
$\{q_0\}$	$\{q_0,q_1\}$	$\{q_0\}$
$\{q_0,q_1\}$	$\{q_0,q_1,q_2\}$	$\{q_0,q_2\}$
$\{q_0,q_1,q_2\}$	$\{q_0,q_1,q_2,q_3\}$	$\{q_0,q_2,q_3\}$
$\{q_0,q_2\}$	$\{q_0,q_1,q_3\}$	$\{q_0,q_3\}$
$\{q_0,q_1,q_2,q_3\}$	$\{q_0,q_1,q_2,q_3\}$	$\{q_0,q_2,q_3\}$
$\{q_0,q_2,q_3\}$	$\{q_0,q_1,q_3\}$	$\{q_0,q_3\}$
$\{q_0,q_1,q_3\}$	$\{q_0,q_1,q_2\}$	$\{q_0,q_2\}$
$\{q_0,q_3\}$	$\{q_0,q_1\}$	$\{q_0\}$

2. $A' =$ 

A 2

1. Gefüllte Tabelle:

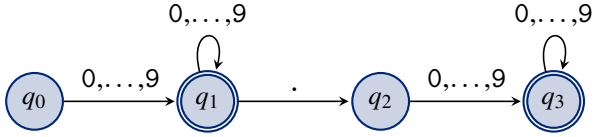
q_1	★			
q_2		★		
q_3	★	★	★	
q_4	★	★	★	★
	q_0	q_1	q_2	q_3

2. Minimaler DEA $A' =$ 

mit $L(A') = L(A)$.

3. $r = \mathtt{(a|b)*abb}$ mit $L(r) = L(A')$

A 3

DEA $A =$ 

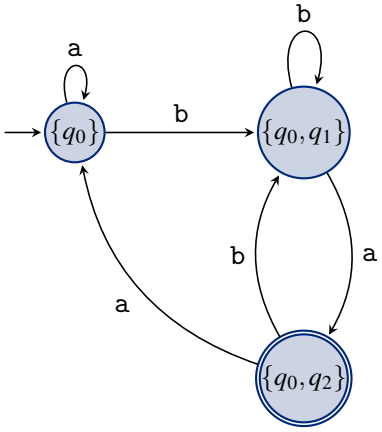
mit $L(r) = L(A)$.

A 4

1.

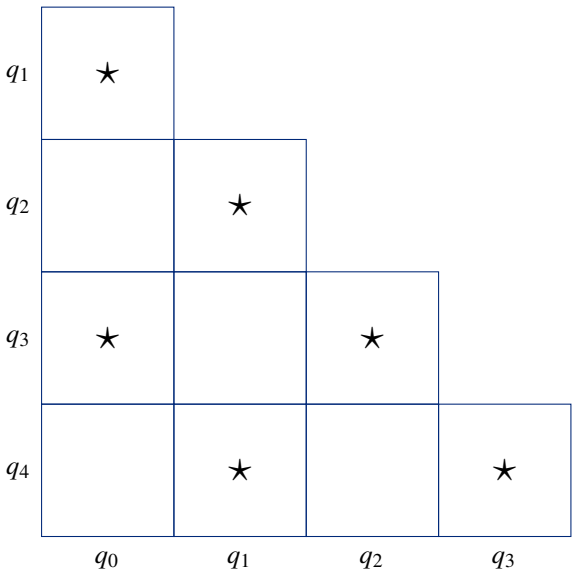
Zustand R	move$(R, \mathtt{a})$	move$(R, \mathtt{b})$
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1\}$
$\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1\}$

2.

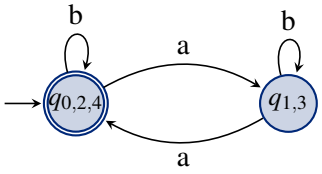


A 5

1.



2.



Teil III

Syntaktische Analyse von Programmiersprachen

Kapitel 6

Parsen von Programmen in TiEL

6.1 Syntaktische Analyse 59
6.2 TiELs Syntax 60
6.2.1 TiELs konkrete Syntax 60
6.2.2 TiELs abstrakte Syntax 60
6.3 Syntaktische Analyse im TiEL-Parser 61
6.4 Syntax mit Grammatiken beschreiben 62
6.4.1 Kontextfreie Grammatiken 62
6.4.2 Präzedenz und Assoziativität 64
6.5 Syntaktische Analyse mit Parsergeneratoren 65
6.5.1 Von Grammatiken zum Parser 65
6.5.2 Scannergenerator Cup 66
6.6 Zusammenfassung 66
Aufgaben mit Lösungen 67

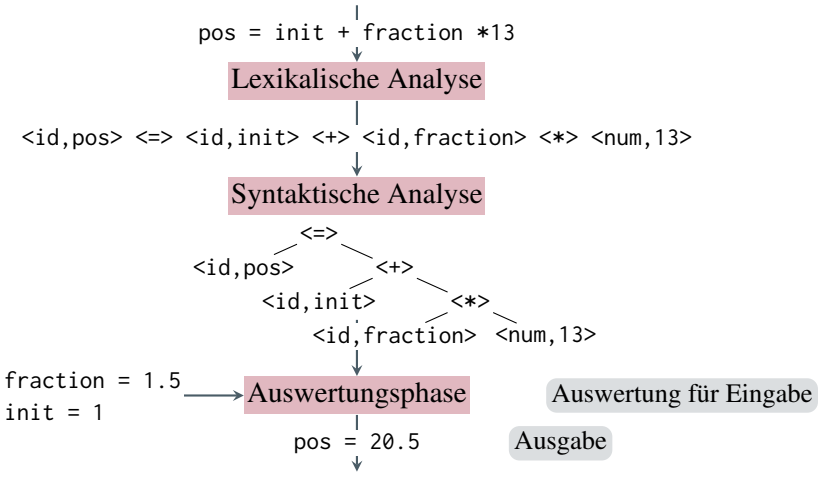
Lernziele

1 Den *TiEL-Parser* als Beispiel eines »handgeschriebenen« Parsers kennen und anpassen können.
2 *Syntaktische Struktur* von Sprachen durch *Grammatiken* beschreiben können.
3 *Ableitungsbäume* und *Syntaxbäume* erstellen können.
4 *Parsergeneratoren* als Alternative zu »handgeschriebenen« Scannern kennen.

6.1 Syntaktische Analyse

Syntaktische Analyse ermittelt hierarchische Struktur

Pipeline eines Interpreters



Definition

Die *Syntaxanalyse* überführt Folge von Grundsymbolen in Syntaxbaum.

6.2 TiELs Syntax

6.2.1 TiELs konkrete Syntax

Konkrete Syntax beschreibt Struktur im Quellprogramm

Definition

Konkrete Syntax beschreibt hierarchischen Aufbau des Programms aus Grundsymbolen.

TiEL-Programm

```
1 fun foo(flag) {
2   if flag { return "Hello World!"; }
3   }
4 print(foo(true));
```

Konkrete Syntax des Programms

- Das Programm besteht aus einer Funktionsdeklaration und einer Anweisung.
- Die Funktionsdeklaration beginnt mit Wortsymbol `fun`. Danach folgt ...
- Die Anweisung beginnt mit dem Bezeichner `print`. Danach folgt ...

6.2.2 TiELs abstrakte Syntax

Abstrakte Syntax beschreibt Struktur der Sprachkonstrukte

Definition

- *Abstrakte Syntax* beschreibt Sprachkonstrukte und lässt für die Auswertung unnötige Bestandteile wie Klammern aus.
- Wird durch *(abstrakten) Syntaxbaum* (engl.: abstract syntax tree; kurz: AST) repräsentiert.

Abstrakte Syntax des TiEL-Programms

```
Generated AST:
(FunctionDeclStmt foo (Params flag) (Body (IfStmt
   (VariableExpr flag) (BlockStmt (ReturnStmt (LiteralExpr
   Hello World!))))))
(ExpressionStmt (CallExpr (VariableExpr print) (CallExpr
   (VariableExpr foo) (LiteralExpr true))))
```

Implementierung der TiEL ASTs

- TiEL ASTs werden intern als Verkettungsstruktur kodiert.
- Klassen sind im Paket `ast` implementiert.
- Knoten des Syntaxbaums werden vom Parser erstellt und vom Auswerter verarbeitet.

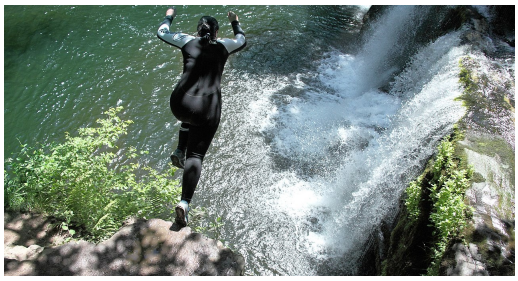
```
public final class IfStmt extends Stmt {

  public final Expr condition;
  public final Stmt thenBranch;
  public final Stmt elseBranch;
```

Klasse `IfStmt`, Klasse IfStmt

6.3 Syntaktische Analyse im TiEL-Parser

Der TiEL-Parser steigt rekursiv ins Programm ab



License: CC BY 2.0, Content: A waterfall diver jumping forwards as much as she can before gravity takes effect, Author: Charlie Marshall from Bristol UK, cropped

Rekursive AST**-Konstruktion**

Methodenaufrufe erkennen geschachtelte Sprachkonstrukte beim *rekursiver Abstieg* (engl. recursive descent). Entspricht einem Aufbau des *abstrakten Syntaxbaums von oben herab* (engl. top-down).

TiEL-Skript ist eine Folge von Deklarationen und Anweisungen

```
  public List<Stmt> parse() {
    var statements = new ArrayList<Stmt>();

    while (!isAtEnd()) {
      statements.add(statement());
    }

    return statements;
  }
```

Klasse `Parser`, Methode parse()

Rekursionsenden konsumieren erwartete Grundsymbole

Rekursionsende
Grundsymbole werden konsumiert; sie führen zu Fallunterscheidungen oder vorgegebene Symbole werden erwartet.

Parser verarbeitet Wortsymbol

```
private Stmt statement() {
  if (match(IF)) return ifStatement();
  if (match(RETURN)) return returnStatement();
  if (match(VAR)) return varDeclaration();
  if (match(WHILE)) return whileStatement();
  if (match(FUN)) return function();
  if (match(LEFT_BRACE)) return block();

  return expressionStatement();
```

Klasse `Parser`, Methode statement()

Ein Rekursionsschritt erkennt geschachtelte Struktur

Rekursionsschritt
Ein Sprachkonstrukt, welches aus Teilen besteht, führt zu geschachtelten Methodenaufrufen. AST-Knoten wird jeweils nach oben zurückgegeben.

Rekursive Abstiege

```
private Stmt ifStatement() {
  var conditions = new ArrayList<Expr>();
  var branches = new ArrayList<Stmt>();
  var positions = new ArrayList<Token.Position>();

  do { // consume available if-elif branches
    positions.add(previous().position());
    conditions.add(expression());
    consume(LEFT_BRACE, "Expect␣'{'␣after␣'if'");
    branches.add(block());
  } while (match(ELIF));
```

Klasse `Parser`, Methode ifStatement()

6.4 Syntax mit Grammatiken beschreiben

6.4.1 Kontextfreie Grammatiken

Grammatiken beschreiben konkrete Syntax

Definition: Grammatik
Eine *(kontextfreie) Grammatik* ist ein Tupel $G = (\Gamma, \Sigma, R, S)$, wobei Γ eine endliche Menge von *Nonterminalen* ist, Σ eine endliche Menge von *Terminalen* mit $\Gamma \cap \Sigma = \emptyset$ ist, R

eine endliche Menge an *Regeln* der Form $X \to u$ mit *Kopf* $X \in \Gamma$ und *Rumpf* u (String über $\Gamma \cup \Sigma$) ist und $S \in \Gamma$ das *Startsymbol* ist.

Konventionen: Grammatiken für die Syntaxanalyse

- Die Terminale in Σ entsprechen den Grundsymbolen.
- Nonterminale und Terminale können auch *term* beziehungsweise **id** sein.
- Regeln wie $A \to ab$ und $A \to aa$ werden als $A \to ab \mid aa$ geschrieben.
- Meist nur Regeln spezifiziert; Tupelschreibweise ist implizit definiert.

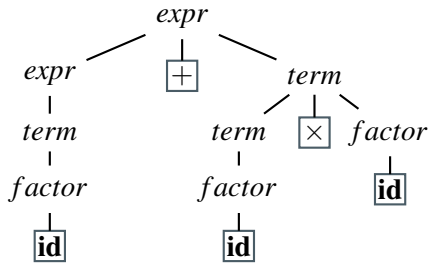
Häufig werden alternative Schreibweisen für Grammatiken verwendet wie die Backus-Naur-Form beziehungsweise deren erweiterte oder angereicherte Variante.

Ableitungsbaum zeigt Struktur basierend auf Grammatik

Beispiel: Grammatik für arithmetische Ausdrücke
Sei $G = (\Gamma, \Sigma, R, A)$ mit $\Gamma = \{term, factor, primary\}$, $\Sigma = \{\mathbf{id}, +, \times, (,)\}$ und R:

$$\begin{aligned} term &\to term + factor \mid factor \\ factor &\to factor \times primary \mid primary \\ primary &\to (term) \mid \mathbf{id} \end{aligned}$$

Beispiel: Ableitungsbaum für arithmetischen Ausdruck
Ableitungsbaum (engl. parse tree)



erzeugt **id** + **id** × **id**

Bemerkung
Variablen werden ohne Kontext ersetzt. Dies ermöglicht Visualisierung als Ableitungsbaum. Syntaxbäume repräsentieren die hierarchische Bedeutung von Wörtern.

Definition
Sei $G = (\Gamma, \Sigma, R, S)$ eine kontextfreie Grammatik. Ein *Ableitungsbaum* (alt. konkreter Syntaxbaum) entsteht, indem wir einen mit S markierten *Wurzelknoten* einführen und danach schrittweise für jeden *Knoten mit einem Nonterminal* X eine Regel $X \to u_1 \dots u_m$ wählen und m *Nachfolgeknoten* an diesen Knoten anhängen, die von links nach rechts mit u_1 bis u_m markiert sind. Falls keine Erweiterung mehr möglich, *ergibt* sich ein Wort an den Blättern.

Nonterminal werden im rekursiven Abstieg zu Methoden

Grammatik und Parser
Ein Nonterminal mit Regeln entspricht einer Methode. Regelrümpfe führen zu Code in der Methode. Es wird entlang der Grammatik von oben abgestiegen.

Analyse von arithmetische Terme im TiEL-Parser

```
private Expr term() {
  var expr = factor();
  while (match(MINUS, PLUS)) {
    var token = previous();
    var op = token.type() == MINUS ?
 BinaryExpr.Operator.SUB : BinaryExpr.Operator.ADD;
    var right = factor();
    expr = new BinaryExpr(expr, op,
 right).withPosition(token.position());
  }
  return expr;
```

Klasse `Parser`, Methode term()

6.4.2 Präzedenz und Assoziativität

Grammatiken kodieren Präzedenz von Operatoren

Beispiel: Präzedenzen in Grammatiken für Ausdrücke

- ohne Präzedenzen:

$$term \rightarrow term + term \mid term \times term \mid (term) \mid \mathbf{id}$$

- mit Präzedenzen

$$\begin{aligned} term &\rightarrow term + factor \mid factor \\ factor &\rightarrow factor \times primary \mid primary \\ primary &\rightarrow (term) \mid \mathbf{id} \end{aligned}$$

Diskussion

Wie sieht ein ungeeigneter Ableitungsbaum bei der Grammatik ohne Präferenzen für $\mathbf{id} + \mathbf{id} \times \mathbf{id}$ aus?

Vorgehensweise: Präzedenzen in Grammatiken

Präzedenz der Operatoren hat über den Ableitungs- und Syntaxbaum Einfluss auf die Semantik eines Programms. Höhere *Präzedenz* bedeutet stärkere *Bindung* des Operators. *Präzedenzebenen* in Grammatik einbauen; Nonterminal für Operator mit höherer Präzedenz steht weiter unten im Ableitungsbaum.

Präzedenzen im TiEL-Parser

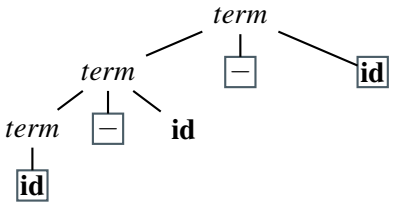
TiEL-Parser implementiert Präzedenzen mit *verschränkten* rekursiven Methoden. Methode `term()` ruft `factor()`, aber nicht ohne konsumiertes `(`-Symbol verschränkt rekursiv umgekehrt.

Grammatiken kodieren Assoziativität von Operatoren

Beispiel: Links- und Rechtsassoziativität

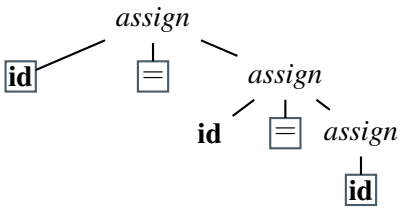
Linksassoziativ Subtraktion

- Grammatik $term \rightarrow term - \mathbf{id} \mid \mathbf{id}$
- Programm $a - b - c$
- Ableitungsbaum



Rechtsassoziative Zuweisung

- Grammatik $assign \rightarrow \mathbf{id} \mid \mathbf{id} = assign$
- Programm $a = b = c$
- Ableitungsbaum



Vorgehensweise: Assoziativität in Grammatiken

Assoziativität gibt die Klammerung für Operatoren vor, die auf der gleichen Präzedenzebene liegen. Assoziativität hat einen Einfluss auf den Ableitungs- und Syntaxbaum und je nach Operation und möglicher Seiteneffekte auf die Auswertung des Programms. Assoziativität wird durch Position des Nonterminals im Regelrumpf festgelegt.

Bemerkung

TiEL-Parser implementiert Assoziativität mit Iteration.

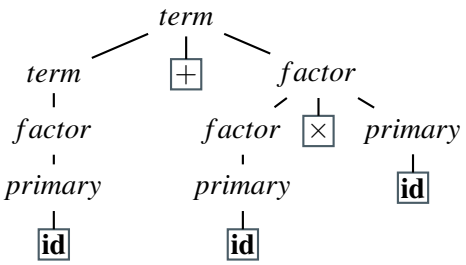
6.5 Syntaktische Analyse mit Parsergeneratoren

6.5.1 Von Grammatiken zum Parser

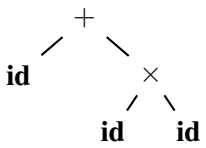
Ableitungsbaum ohne »Helferknoten« führt auf Syntaxbaum

Beispiel

Ableitungsbaum



Syntaxbaum (schematisch)



Bemerkung

Ableitungsbaum für konkrete Syntax wird häufig nicht explizit erstellt. Ausgabe der syntaktischen Analyse ist der Syntaxbaum für abstrakte Syntax. Implementierung wird vereinfacht, falls sich Ableitungs- und Syntaxbaum ähneln.

Parsergeneratoren erzeugen Parser aus Grammatik

Parsergeneratoren

Ein *Parsergenerator* erhält als Eingabe eine *Konfigurationsdatei*. Diese wird eingelesen und in einen Parser übersetzt. Scannergeneratoren sind Compiler für Parser.

Bestandteil der Konfiguration I: Grammatik

Konfiguration enthält eine Grammatik für die konkreten Syntax der Quellsprache.

Bestandteil der Konfiguration II: Attribute

Die Regeln der Grammatik werden mit *Attributen* versehen; dies ist meist Programmcode in der Implementierungssprache zur Generierung des AST.

6.5.2 Scannergenerator Cup

Cup-Beschreibung enthält konkrete und abstrakte Syntax

Beschreibung für Parsergenerator Cup

Konkrete Syntax Wird durch kontextfreie Grammatik beschrieben.
Abstrakte Syntax Entsteht aus auszuführendem Code für Regeln.

Ausschnitt aus Beschreibung eines TiEL-Parsers in Cup

```
116 term ::= term:lhs PLUS:op factor:rhs {: RESULT = new
       BinaryExpr(lhs, (Token) op, rhs); :}
117     | term:lhs MINUS:op factor:rhs {: RESULT = new
          BinaryExpr(lhs, (Token) op, rhs); :}
118     | factor:expr {: RESULT = expr; :};
119
120 factor ::= factor:lhs STAR:op unary:rhs {: RESULT = new
       BinaryExpr(lhs, (Token) op, rhs); :}
121     | factor:lhs SLASH:op unary:rhs {: RESULT = new
          BinaryExpr(lhs, (Token) op, rhs); :}
122     | unary:expr {: RESULT = expr; :};
```

6.6 Zusammenfassung

- Die *syntaktische Analyse* wird vom *Parser* durchgeführt und erstellt einen *abstrakten Syntaxbaum* gültiger Programme.
- Der *abstrakte Syntaxbaum* beschreibt die *abstrakte Syntax* eines Programms, welches die *Sprachkonstrukte*, aber keine »helfenden« Elemente wie beispielsweise Klammern enthält.
- *Kontextfreie Grammatiken* definieren die *konkrete Syntax* eines Programms, welche basierend auf den Grundsymbolen der Sprache den Aufbau von Programmen beschreibt.
- Parser erstellen *Ableitungsbäume* der konkreten Syntax meist nur implizit; im TiEL-Parser entspricht dies den rekursiven Methodenaufrufen.
- Grammatiken helfen, um Syntax eindeutig herauszuarbeiten und zu kommunizieren.
- Zudem werden Grammatiken annotiert mit *Attributen* für die Generierung von Parsern mit *Parsergeneratoren* verwendet.

Literatur

[1] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021. (Kapitel 5 und 6)

[2] A. Aho, M. Lam, R. Sethi und J. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2006. (Abschnitte 2.1 bis 2.5 und 4.1 bis 4.4 und 4.9)

[3] Handpeter Mössenböck. *Compilerbau: Grundlagen und Anwendungen*. dpunkt.verlag, 2024. (Kapitel 3 und Abschnitte 7.2 bis 7.6)

[4] Uwe Meyer. *Grundkurs Compilerbau*. Rheinwerk Computing, 2021. (Kapitel 4 ohne Bottom-Up Parsing und Kapitel 5)

Aufgaben

A 1 (Grammatik analysieren)

Wie betrachten die Grammatik mit den folgenden Regeln:

$$S \to SS\texttt{+} \mid SS\texttt{*} \mid \texttt{a}$$

1. Konstruieren Sie einen Ableitungsbaum mit der obigen Grammatik für den String `aa+a*`.
2. Beschreiben Sie die Sprache der Strings, die durch die obige Grammatik erzeugt werden.

A 2 (Grammatiken erstellen)

Geben Sie für jede der folgenden Sprachen eine kontextfreie Grammatik an, die genau die Strings dieser Sprache erzeugt. Begründen Sie jeweils die Korrektheit der Grammatik kurz.

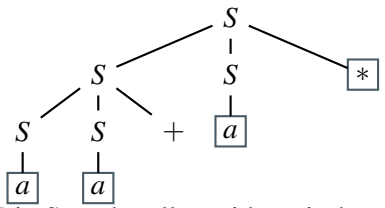
1. $\{w \in \{0,1\}^* \mid w \text{ enthält gleich viele 0en und 1en}\}$
2. $\{\texttt{a}^k\texttt{b}^m\texttt{c}^n \mid k = m \text{ oder } m = n\}$

A 3 (Grammatik für TiEL-Ausdrücke erstellen)

TiEL erlaubt verschiedene Operatoren in Ausdrücken, die jeweils eine festgelegte relative Präzedenz und Assoziativität haben. Stellen Sie die Regeln einer kontextfreie Grammatik auf, die genau die Ausdrücke ohne Funktionsaufrufe in TiEL erzeugt. Verwenden Sie für Ihre Recherche die Implementierung `Parser.java`.

Lösungen

A 1

1. 

2. Die Sprache aller arithmetischen Ausdrücke mit Operatoren $+$ und $*$ und dem Bezeichner a. Diese Ausdrücke werden in Postfixnotation aufgeschrieben.

A 2

1. $$S \to S0S1S \mid S1S0S \mid \lambda$$

   Wenn eine 0 generiert wird, dann wird immer auch eine 1 generiert. Dies geschieht in beliebiger Reihenfolge. »Überschüssige Variablen S werden am Ende eliminiert.
2. $$\begin{aligned} S &\to TU \mid VW \\ T &\to \lambda \mid \mathtt{a}T\mathtt{b} \\ U &\to \lambda \mid Uc \\ W &\to \lambda \mid \mathtt{b}W\mathtt{c} \\ V &\to \lambda \mid Va \end{aligned}$$

   Die Sprache kann man als Vereinigung zweier Sprachen ansehen, so wie die Verknüpfung der Eigenschaften mit dem oder. Die Grammatik ist entsprechend aufgebaut und »entscheidet« am Anfang für eine der beiden Sprachen.

A 3

$$\begin{aligned} \mathit{expr} &\to \mathit{assign} \\ \mathit{assign} &\to \mathit{or} = \mathit{assign} \mid \mathit{or} && \text{(rechtsassoziativ)} \\ \mathit{or} &\to \mathit{or}\ \mathbf{or}\ \mathit{and} \mid \mathit{and} && \text{(linksassoziativ)} \\ \mathit{and} &\to \mathit{and}\ \mathbf{and}\ \mathit{equality} \mid \mathit{equality} \\ \mathit{equality} &\to \mathit{equality} == \mathit{comparison} \mid \mathit{comparison} \\ \mathit{comparison} &\to \mathit{comparison} < \mathit{term} \mid \mathit{term} \\ \mathit{term} &\to \mathit{term} + \mathit{factor} \mid \mathit{term} - \mathit{factor} \mid \mathit{factor} \\ \mathit{factor} &\to \mathit{factor} * \mathit{unary} \mid \mathit{factor} / \mathit{unary} \mid \mathit{unary} \\ \mathit{unary} &\to \mathbf{not}\ \mathit{unary} \mid -\mathit{unary} \mid \mathit{call} \\ \mathit{call} &\ldots \end{aligned}$$

Teil IV

Auswerten von Programmiersprachen

Teil V

Theorie der Implementierung von Programmiersprachen